BOPIS: Bayesian Optimization and Pareto-Based Intelligent Configuration Selection for Energy-Efficient Local LLM Inference

A Thesis
Presented to the Faculty of the
College of Computer and Information Sciences
Polytechnic University of the Philippines

In Partial Fulfilment of the Requirements for the Degree
Bachelor of Science in Computer Science

De Guzman, Aaron Bien
Patacsil, Harold
Piastro, Lance Allen

October 2026

TABLE OF CONTENTS

The Problem and Its Setting

Introduction……3

Theoretical Framework……5

Transformer Architecture and Computational Complexity……5

LLM Quantization and Numerical Precision……6

Hardware-Aware Configuration……7

Bayesian Optimization and Gaussian Process Surrogate Modeling……8

Pareto Multi-Objective Optimization……11

Conceptual Framework……12

Statement of the Problem……14

Scope and Limitation of the Study……15

Significance of the Study…… 18

Definition of Terms…… 19

Review of Literature and Studies

Related Literature and Studies……28

Local LLM Inference and The Need for Efficient Configuration Selection……28

Energy Consumption and Inference Efficiency in Large Language Models……29

Configuration Parameters Affecting Local LLM Performance……33

Bayesian Optimization for LLM Inference Parameter Tuning……34

LLM Quantization and Numerical Precision……38

Multi-Objective Optimization and Pareto Analysis in LLM Systems……41

Output Quality Evaluation using BERTScore F1……45

Related Systems and Optimization Frameworks……46

Synthesis of the Study……47

Methodology

Research Design……52

Sources of Data…… 54

Sampling Data……56

System Architecture…… 57
Research Instrument……..75
Data Generation/Gathering Procedure……..78
Ethical Considerations…… 82
Data Analysis (Procedure and Treatment).....83
Statistical Treatment……89

References……..94
Appendices……...100

Chapter 1
THE PROBLEM AND ITS SETTING

Introduction

Artificial Intelligence (AI) has become one of the most transformative fields in modern computing, enabling advancement in automation, data analysis, and human-computer interaction. Among the most impactful developments are large language models (LLMs), which are built on transformer architectures (Vaswani et al., 2023), and are capable of performing complex natural language tasks such as text generation, summarization, and question answering. These models are now widely integrated into real-world applications, including intelligent assistance, productivity tools, and decision-support systems (OpenAI, 2023; Zhao et al., 2023).

Despite these advancements in scale and capability, they also introduce a challenge: energy consumption. Inference is the process by which a trained LLM generates a response to a given input, and it is the phase that end users directly experience. Unlike model training, inference occurs repeatedly and continuously, making its cumulative resource consumption a pressing concern in environments where hardware capacity and energy availability are constrained (Patterson et al., 2021; Strubell et al., 2019). The energy consumed during inference, the speed at which responses are generated, and the quality of those responses are directly influenced by runtime configuration settings such as input token length, batch size, GPU layer offloading, and CPU thread allocation. Different combinations of these parameters yield substantially different performance outcomes, yet most deployments rely on default configurations without systematic evaluation.

The primary motivation for deploying LLMs locally rather than relying solely on cloud-based AI services is data privacy and infrastructure control. Organizations in sectors such as finance, healthcare, education, and legal services may need to ensure that sensitive prompts

and generated responses remain within their own controlled environment (Yan et al., 2025). However, local deployment also transfers the burden of computational efficiency to the organization, making energy consumption, inference speed, output quality, and hardware usage important practical concerns (Husom et al., 2024; Patterson et al., 2021). For this reason, the study focuses on optimizing local LLM inference configurations using a standardized instruction-following dataset that reflects common LLM tasks without requiring domain-specific expertise.

Existing research has made progress in measuring and characterizing LLM inference efficiency, but significant gaps remain. Profiling studies such as Husom et al. (2024) and benchmarking frameworks such as TokenPowerBench (Niu et al., 2025) provide structured methods for measuring energy usage, but they focus on reporting rather than improving. Optimization studies, on the other hand, often target a single metric such as latency or throughput without simultaneously addressing energy consumption and output quality. Furthermore, comparative evaluation in the literature frequently involves different hardware environments, model families, and measurement tools, making it difficult to draw fair conclusions about which approach is genuinely better for a given local deployment scenario (Zhou et al., 2024). These limitations show the need for a system that can both select an efficient configuration and validate whether the optimization process used to reach that configuration is reliable and sample-efficient.

The study addresses these gaps by proposing BOPIS -- Bayesian Optimization and Pareto-Based Intelligent Configuration Selection -- a system that treats local LLM inference configuration as a decision problem. The system chooses among possible configurations based on measurable trade-offs in energy consumption, inference speed, output quality, and resource utilization. In the study, a configuration is considered optimized when it reduces energy

consumption relative to the baseline while maintaining acceptable inference speed and output quality, measured through tokens per second and BERTScore F1.

Theoretical Framework

The study is grounded in three interconnected theoretical foundations that collectively explain why LLM inference configuration matters, how the configuration space can be searched efficiently, and how the best trade-off among competing performance objectives can be identified. These foundations are: transformer architecture and computational complexity, Bayesian Optimization with Gaussian Process surrogate modeling, and Pareto multi-objective optimization.

Transformer Architecture and Computational Complexity

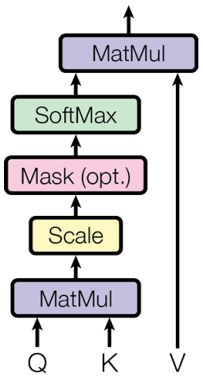


Figure 1.1: Scaled-Dot Product Attention by Vaswani et al. (2023)

Vaswani et al. (2023) introduced the transformer architecture, which underlies all modern large language models. As shown in Figure 1.1, the core mechanism of the transformer is scaled dot-product attention, defined as $Attention(Q, K, V) = softmax(QK^T/\sqrt{d_k})V$, where $Q, K, and\, V$ represent the query, key, and value matrices, and $\sqrt{d_k}$ normalizes the dot products.

This mechanism computes relationships between every pair of input tokens through matrix multiplications, enabling the model to capture long-range contextual dependencies.

A critical consequence of this design is that computational complexity scales quadratically with input sequence length; that is, doubling the number of input tokens roughly quadruples the number of attention operations required. This scaling behavior means that configuration parameters such as context window size, batch size, and numerical precision have a direct and non-linear effect on energy consumption, inference speed, and memory usage. Understanding this relationship is what motivates treating inference configuration as an optimization problem: small parameter changes can produce large differences in resource demand, and there is no single universal configuration that minimizes all costs simultaneously.

LLM Quantization and Numerical Precision

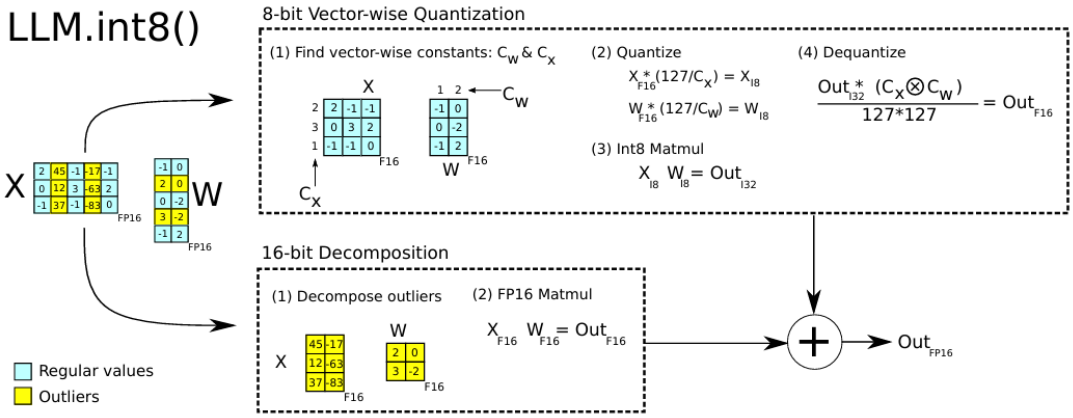


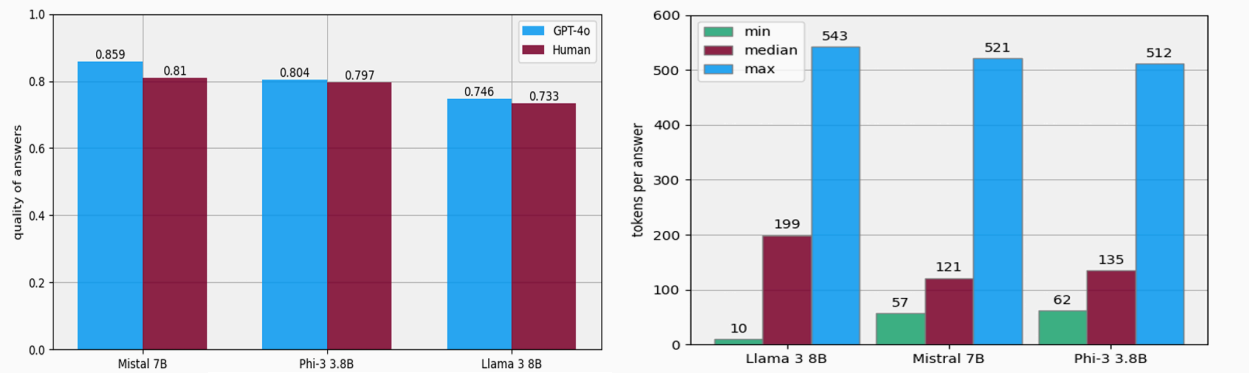
Figure 1.2. Vector-Wise Quantization and Mixed-Precision Decomposition in LLM.int8() by Dettmers et al (2022)

Dettmers et al. (2022) introduced LLM.int8() as an efficient quantization technique that reduces the computational and memory cost of large language model inference while preserving numerical stability. As shown in Figure 1.2, the method applies vector-wise 8-bit quantization to the majority of model weights and activations by computing scaling constants

and performing integer matrix multiplication followed by dequantization. To address the loss of precision caused by quantization, a mixed-precision decomposition is used in which outlier values are separated and processed in higher precision (F16), then recombined with the quantized results. This approach enables significant efficiency gains without substantial degradation in model accuracy.

In the context of the study, LLM.int8() demonstrates how low-level numerical optimization directly affects system-level metrics such as energy consumption and inference latency. Since BOPIS evaluates configurations involving precision formats and model parameters, quantization becomes a crucial factor in reducing energy usage while maintaining acceptable output quality. The incorporation of this highlights the importance of treating LLM inference as a system optimization problem, where hardware efficiency and algorithmic design jointly influence performance outcomes.

Hardware-Aware Configuration



model	per token	per question	total
Llama 3 8B	1,76 mWh	364,61 mWh	18.230,88 mWh
Mistral 7B	1,55 mWh	266,77 mWh	13.338,30 mWh
Phi-3 3,8B	0,93 mWh	162,94 mWh	8.146,94 mWh

Figure 1.3. Energy, Latency, and Quality Trade-offs in Local LLM Deployment by Bast et al. (2024)

Bast et al. (2024) examined the trade-offs between energy consumption, inference latency, and output quality across different locally deployed language models. As illustrated in Figure 1.3, smaller models such as Phi-3 3.8B exhibit lower energy consumption per token and per query compared to larger models like Llama 3.8B, while maintaining competitive quality scores. Additionally, latency distributions show that larger models generate longer responses and require more tokens per answer, contributing to higher total energy usage. These results highlight the inherent trade-offs between model size, computational cost, and performance quality.

In the study, these trade-offs form the foundation of the optimization problem addressed by BOPIS. The system must balance competing objectives; minimizing energy consumption while ensuring that inference speed and output quality remains within acceptable thresholds. The variability observed across models reinforces the need for systematic configuration search rather than relying on default settings. By integrating Bayesian Optimization and Pareto analysis, BOPIS is able to navigate these trade-offs effectively, identifying configurations that achieve efficient energy usage without significantly compromising response quality or latency.

Bayesian Optimization and Gaussian Process Surrogate Modeling

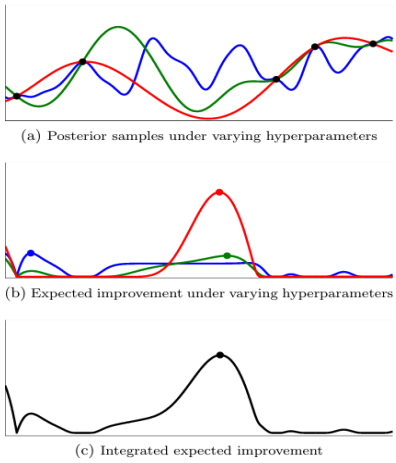


Figure 1.4. Gaussian Process Surrogate and Acquisition Function in Bayesian Optimization by Snoek et al. (2012)

Snoek et al. (2012) established Bayesian Optimization as a principled approach for optimizing expensive black-box functions; systems where the relationship between inputs and outputs cannot be analytically derived, and each evaluation carries a high cost. As shown in Figure 1.4, Bayesian Optimization builds a Gaussian Process (GP) surrogate model from observed evaluation data. The GP approximates the unknown objective function by providing a predicted mean $\mu(x)$ and an uncertainty estimate $\sigma(x)$ for any unevaluated configuration. An acquisition function, specifically Expected Improvement, is defined as $EI(x) = E[max(0, f(x) - f(x^{+}))]$ one that selects the next configuration to evaluate by balancing exploration of uncertain regions with exploitation of known high-performing areas.

In the study, LLM inference is treated as a black-box function. The relationship between configuration parameters and system-level outcomes such as energy consumption, inference speed, and output quality is non-linear, hardware-dependent, and expensive to evaluate exhaustively. Bayesian Optimization enables BOPIS to identify promising configurations with far fewer evaluations than random or grid search, which is the fundamental advantage over the AutoTuner-inspired random search baseline used for comparison in this study. Wang et al. (2021) further demonstrated that pre-trained Gaussian Processes can accelerate this search by providing better initialization, which informs the surrogate model strategy used in BOPIS.

The reliability of the Gaussian Process surrogate model can also be evaluated through prediction accuracy metrics. Since the surrogate model estimates the performance of unevaluated configurations, its predictions must be compared with actual measured outcomes. Wilkins et al. (2024) demonstrated that energy and runtime behavior in LLM inference can be modeled with high predictive accuracy, supporting the use of prediction error as a validation measure. In the study, Mean Absolute Error (MAE) and Normalized Prediction Error (NPE) are used to compare GP-predicted energy values with actual measured energy values, allowing the study to determine whether the surrogate model reliably guides the search process.

The efficiency of Bayesian Optimization is further evaluated through convergence and sample efficiency. Convergence is tracked using the best energy value found per iteration and the improvement per iteration, represented as ΔE. Sample efficiency is measured using the Sample Efficiency Ratio (SER), which compares the iteration at which BOPIS and random search first identify the final Pareto-optimal configuration. Since random search is widely used as an uninformed baseline for hyperparameter optimization (Bergstra & Bengio, 2012), and Bayesian-guided search has been shown to reduce optimization cost compared with random search in LLM-based systems (Sabbatella, 2025), SER helps determine whether Bayesian Optimization provides practical value over unguided search under the same evaluation budget.

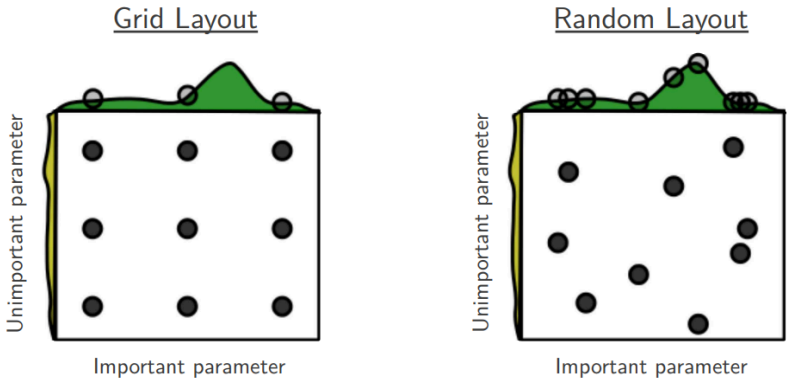


Figure 1.5. Grid and Random Search Comparison by Bergstra and Bengio (2012)

Bergstra and Bengio (2012) demonstrated that random search is a practical and reproducible method for hyperparameter optimization. As shown in Figure 1.5, random search explores more distinct values of important parameters compared with grid search when only some dimensions strongly affect performance. In the study, Random Search is used as a baseline configuration selection method because it evaluates the same search space as BOPIS without model-guided decision-making. This allows the study to determine whether the Bayesian Optimization and Pareto-based selection strategy of BOPIS provides improvement over an uninformed search method under the same dataset, hardware, and evaluation metrics.

In BOPIS, Pareto analysis is applied to the configurations identified by Bayesian Optimization. Rather than selecting a single configuration based on an arbitrary weighting of objectives, the Pareto front presents the complete set of non-dominated trade-off solutions across energy consumption, inference speed, and output quality. The final recommended configuration is selected from this Pareto front as the decision output of BOPIS; the configuration that achieves the greatest reduction in energy consumption while satisfying the minimum acceptable thresholds for inference speed and output quality. This approach is what distinguishes BOPIS from random search baselines, which have no mechanism for identifying or evaluating trade-offs systematically.

Pareto Multi-Objective Optimization

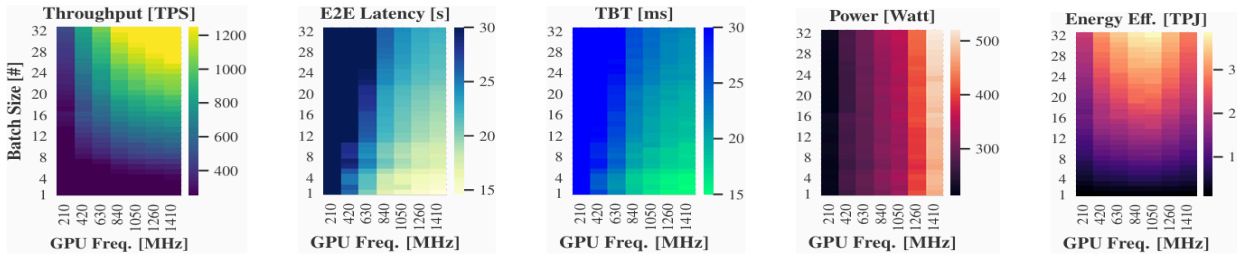


Figure 1.6. Trade-offs Between Energy Efficiency and Performance by Kakolyris et al. (2024)

Optimizing for a single metric in isolation produces configurations that may be impractical. A configuration that minimizes energy consumption may generate responses too slowly to be useful; one that maximizes speed may sacrifice output quality. As demonstrated by Kakolyris et al. (2024) and illustrated in Figure 1.6, energy efficiency and performance metrics are competing objectives whose optimal values cannot always be achieved simultaneously. Pareto Optimization addresses this by identifying the set of configurations in which no objective can be improved without worsening another; formally, a configuration x^* is Pareto-optimal if there exists no x such that $f_i(x) \leq f_i(x^*)$ for all objectives i and $f_j(x) < f_j(x^*)$ for at least one objective j.

Conceptual Framework

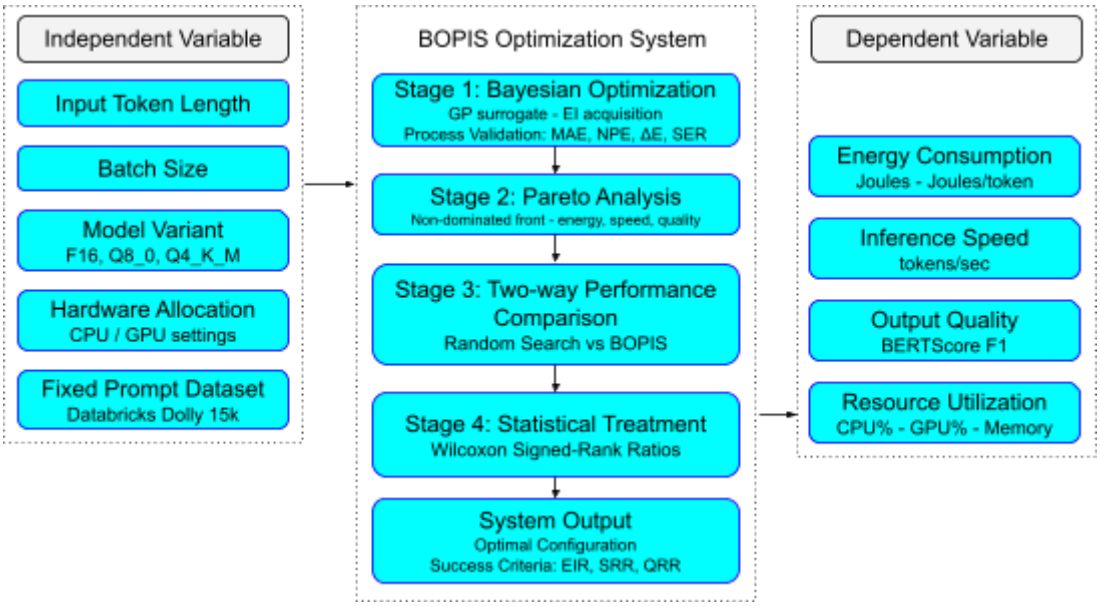


Figure 1.7: Conceptual Framework of BOPIS: Bayesian Optimization and Pareto-Based Intelligent Configuration Selection for Energy-Efficient Local LLM Inference

The conceptual framework of the study illustrates the relationship between the independent variables, the BOPIS optimization system, and the dependent variables, as shown in Figure 1.7. The independent variables are the configurable parameters of local LLM inference that define the search space explored by the system. These include input token length, batch size, precision/quantization variant, GPU layer offloading, and CPU thread allocation. The study uses the same base model, Qwen2.5-1.5B-Instruct, across three GGUF variants: F16, Q8_0, and Q4_K_M. The prompt dataset, drawn from Databricks Dolly 15k, is used as a fixed and standardized input across all conditions to ensure fair comparison.

The study treats local LLM inference configuration as a decision problem, where the system must choose among possible configurations based on measurable trade-offs in energy consumption, inference speed, output quality, and resource utilization.

These variables are fed into the BOPIS optimization system, which operates through three sequential stages. In the first stage, Bayesian Optimization searches the configuration

space using a Gaussian Process surrogate model, identifying configurations that perform well across the three primary performance dimensions with minimal evaluations. In the second stage, Pareto analysis evaluates the trade-offs among the promising configurations identified by Bayesian Optimization, producing a Pareto front of non-dominated solutions from which the final recommended configuration is selected. In the third stage, the BOPIS-optimized configuration is compared against a random search baseline configuration, both tested under the same hardware environment and prompt dataset.

The dependent variables represent the measurable outcomes evaluated for each configuration across both comparison conditions : energy consumption measured in Joules and energy per token, inference speed measured in tokens per second, output quality measured through proxy-based evaluation metrics, and resource utilization measured as CPU utilization percentage, GPU utilization percentage, and memory usage. The final output of the system is a statistically validated configuration recommendation; a concrete decision output that identifies which approach produces the most energy-efficient performance without sacrificing acceptable inference speed and output quality. A configuration is considered optimized when it achieves lower energy consumption than the random search baseline while maintaining inference speed and BERTScore F1 at or above their baseline values.

To determine whether the final system output is considered optimized, the study uses three configuration success indicators: Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR). EIR verifies whether the BOPIS-recommended configuration reduces energy consumption, while SRR and QRR verify whether inference speed and output quality are retained within acceptable thresholds. These indicators ensure that the final recommendation is not selected based on energy reduction alone, but on a balanced improvement that preserves practical response performance and semantic output quality. The

detailed formulas and threshold values for these ratios are discussed in Chapter 3 under Data Analysis.

Statement of the Problem

The study aims to design, implement, and evaluate BOPIS, a Bayesian Optimization and Pareto-Based Intelligent Configuration Selection system, for locally deployed Large Language Model (LLM) inference. BOPIS treats local LLM inference configuration as a decision problem, selecting among possible configurations based on measurable trade-offs in energy consumption, inference speed, and output quality. A configuration is considered optimized when it achieves lower energy consumption while maintaining acceptable inference speed and BERTScore F1 output quality. Its performance is evaluated through a two-way comparison.

The study seeks to answer the following questions:

1. What is the performance of the random search baseline configuration in terms of:
    a. Energy consumption, measured in Joules and Joules per token (J/token);
    b. Inference speed, measured in tokens per second (tokens/sec);
    c. Output quality, measured using BERTScore F1; and
    d. Resource utilization, measured as CPU utilization (%), GPU utilization (%), and memory usage (mb)?
2. What is the performance of the BOPIS-optimized configuration in terms of:
    a. Energy consumption (Joules, J/token);
    b. Inference speed (tokens/sec);
    c. Output quality (BERTScore F1);
    d. Resource utilization (CPU%, GPU%, memory usage)? And;
    e. Configuration improvement as measured by Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR)?

3. Is there a statistically significant difference as determined by the Wilcoxon signed-rank test between random search baseline, and BOPIS-optimized configurations in terms of:
    a. Energy consumption (Joules, J/token);
    b. Inference Speed (tokens/sec);
    c. Output quality (BERTScore F1); and
    d. Resource utilization (CPU%, GPU%, memory usage); and
    e. Per-variant performance, as measured across F16, Q8_0, and Q4_K_M GGUF precision/quantization variants in terms of energy consumption, inference speed, and output quality?

Scope and Limitation of the Study

The study designs and evaluates BOPIS, a configuration selection system for locally deployed Large Language Model (LLM) inference. Using Bayesian Optimization and Pareto-based trade-off analysis, the system identifies an energy-efficient inference configuration and compares its performance against random search baseline. All two conditions are tested using the same hardware environment, the same locally deployed model, and the same standardized prompt dataset from Databricks Dolly 15k.

The base model used in the study is Qwen2.5-1.5B-Instruct, served through llama.cpp using three GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M. The study's original instrument specified Qwen2.5-1.5B; that model was replaced after hardware profiling established that no configuration of it satisfies the system-memory guard on the study host, and that even its most compressed feasible variant admits no GPU-layer offload. Qwen2.5-1.5B-Instruct admits the full GPU-layer domain and a feasible configuration space large enough to execute the five-parameter search of Table 3.2 as specified. The substitution is disclosed as a deliberate instrument change rather than presented as the original design. These

variants represent different deployment formats of the same base model and are evaluated to determine how precision and quantization affect energy consumption, inference speed, output quality, and resource utilization. All inference calls are issued through llama.cpp's OpenAI-compatible server endpoint with temperature set to 0.0, selecting greedy decoding so that sampling contributes no variance across experimental conditions. Bitwise-identical output across configurations is not claimed: floating-point addition is not associative, and thread count, batch size, and backend selection alter the order in which partial sums are reduced (Goldberg, 1991). Decoding is therefore deterministic given a fixed configuration, while output may differ between configurations for reasons unrelated to precision alone. The study is limited to the tested model variants, inference backend, hardware environment, and configuration space used during experimentation. The F32 (full-precision) variant is excluded from the configuration space, as it requires VRAM substantially exceeding what the available hardware supports and is not representative of practical local LLM deployment, where quantized and half-precision formats are the standards.

Performance is assessed in terms of energy consumption, inference speed, and output quality, with resource utilization treated as a supporting metric. Energy consumption is measured in Joules and Joules per token, inference speed is measured in tokens per second, output quality is measured using BERTScore F1, and resource utilization is measured through CPU utilization, GPU utilization, and memory usage. Configuration evaluation is limited to inference parameters supported by the available hardware and llama.cpp backend, including input token length, batch size, precision/quantization variant, GPU layer offloading, CPU thread allocation, and other supported runtime settings.

In addition to evaluating the final recommended configuration, the study also assesses the reliability and efficiency of the Bayesian Optimization process. Gaussian Process surrogate model accuracy is evaluated using Mean Absolute Error (MAE) and Normalized Prediction Error

(NPE), convergence behavior is evaluated through the best energy value found per iteration and improvement per iteration (ΔE), and sample efficiency is evaluated using the Sample Efficiency Ratio (SER) against random search. The final configuration is further interpreted using Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR) to determine whether energy reduction is achieved while maintaining acceptable inference speed and output quality.

The study does not assume that the BOPIS-optimized configuration will outperform the random search baseline in all cases. If EIR is negative, the result indicates that the BOPIS-recommended configuration consumed more energy under the tested conditions. If SRR falls below the required threshold, the result indicates that any energy reduction was achieved at the cost of unacceptable inference slowdown. If QRR falls below the required threshold, the result indicates that the optimized configuration did not preserve acceptable semantic output quality. In such cases, the configuration will not be classified as successfully optimized, and the result will be reported as a null, negative, or partial outcome rather than treated as evidence of improvement. Hardware profiling enumerates accelerators through NVML, which reports only NVIDIA devices. The study host additionally exposes an Intel Iris Xe integrated GPU with greater available memory than the discrete card; it was outside the instrument's detection scope and is not included in the feasible configuration space. Extending device discovery beyond NVML is identified as future work.Offloading transformer layers to the study host's discrete GPU reduced generation throughput monotonically in the number of layers offloaded, to 0.28x of CPU-only throughput at full offload. The device is a GP108-class part with a 64-bit memory bus; because decode is memory-bandwidth-bound, offloading moves work to the slower of the two available memory paths. Accordingly, GPU layer count is retained as a search dimension but is expected to be optimized toward zero on this hardware class. This conclusion is specific to this class of discrete GPU and does not generalize to parts with wider memory buses.

The scope is limited to inference-level configuration optimization. Model architecture, model weights, training procedures, fine-tuning, and cloud or distributed deployment environments are excluded. The study does not modify the model architecture or retrain the model. Energy consumption is estimated using software-based monitoring tools, which may introduce measurement variance due to polling intervals, driver-level reporting limitations, and background system activity. Output quality is assessed using BERTScore F1, which measures semantic similarity between generated and reference responses but does not fully capture factual accuracy, coherence, or task-specific correctness. Findings are applicable only to the tested model file, hardware environment, configuration space, monitoring procedure, and prompt dataset used in the study.

Significance of the Study

The study presents an optimization-based approach for improving the efficiency of locally deployed Large Language Model (LLM) inference through Bayesian Optimization and Pareto-based Intelligent Configuration Selection (BOPIS). The study benefits students and researchers, software developers and system engineers, organizations, and future researchers.

Students and Researchers

The study can help students and researchers in computer science, machine learning, and green computing understand how LLM inference performance can be evaluated and optimized in a controlled local environment. By using a fixed dataset, consistent metrics, statistical comparison, and Bayesian Optimization process-validation measures, the study provides a structured basis for analyzing energy consumption, inference speed, output quality, resource utilization, and optimization reliability.

Software Developers and System Engineers

The study can be useful to software developers and system engineers who deploy LLMs locally by providing a practical basis for choosing efficient inference configurations. Instead of

relying only on default settings or trial-and-error testing, the BOPIS system can help identify configurations that reduce energy use while maintaining acceptable response performance.The inclusion of configuration success indicators also helps practitioners determine whether an optimized configuration provides meaningful improvement over the default setup.

Organizations

The study can benefit organizations and businesses by supporting cost-efficient and resource-aware for local LLM deployment. By reducing unnecessary energy usage and hardware resource consumption while maintaining acceptable performance, the study may help lower operational costs, support better IT resource planning, and make local AI adoption more practical for businesses with limited computing resources. The system may also assist organizations that prefer local deployment due to data privacy and infrastructure control requirements.

Future Researchers

The study can serve as a foundation for research on energy-efficient AI deployment and local LLM configuration selection. Future researchers may extend the BOPIS approach by applying it to other LLMs, larger datasets, different hardware environments, or real-time adaptive configuration selection. They may also expand the evaluation approach by improving the Bayesian Optimization reliability metrics, testing additional quality measures, or comparing BOPIS against other configuration selection methods.

Definition of Terms

Large Language Model (LLM) - A transformer-based deep learning model trained on large-scale text data to perform natural language tasks such as text generation, summarization, and question answering. In the study, the LLM is deployed locally on a single-machine CPU-GPU setup.

Local LLM Inference - The process of running a large language model on local hardware to generate output tokens from an input prompt. In the study, inference is performed locally using llama.cpp rather than through a cloud-based AI service.

Inference Configuration - The set of runtime parameters that control how the LLM performs inference. In the study, these include input token length, batch size, precision/quantization variant, GPU layer offloading, CPU thread allocation, and other supported llama.cpp runtime settings.

Configuration Space - The set of all feasible inference configurations evaluated by BOPIS. Each configuration represents one possible combination of runtime settings, including input token length, batch size, precision/quantization variant, GPU layer offloading, and CPU thread allocation, that may affect energy consumption, inference speed, output quality, and resource utilization.

Precision/Quantization Variant - The model deployment format used during local LLM inference, referring to the numerical representation of the model weights in GGUF format. In the study, the evaluated precision/quantization variants are F16, Q8_0, and Q4_K_M. These variants are treated as part of the configuration search space because they may affect energy consumption, inference speed, memory usage, and output quality.

Numerical Precision - The bit-width format used to represent model weights and computations during inference. In the study, numerical precision is operationalized through GGUF precision/quantization variants rather than through training-time precision modification.

Token - The basic unit of text processed by a large language model. A token may represent a word, subword, symbol, or character sequence depending on the model tokenizer. In the study, energy consumption is normalized as Joules per token, and inference speed is measured in tokens per second.

Prompt - The input text submitted to the LLM for response generation. In the study, prompts are sampled from the Databricks Dolly 15k dataset and used consistently across the compared configurations.

500-Prompt Evaluation Dataset - The fixed stratified sample of 500 prompts selected from Databricks Dolly 15k for final performance evaluation. This dataset is used to compare the random search baseline and the BOPIS-recommended configuration.

50-Prompt Proxy Subset - A smaller stratified subset selected from the 500-prompt evaluation dataset. In the study, this subset is used during Bayesian Optimization and random search configuration search to reduce computational cost before final evaluation on the full 500-prompt dataset.

Energy Consumption - The amount of electrical energy used during LLM inference, measured in Joules. In the study, energy consumption is also normalized as Joules per token to allow fair comparison across responses of different lengths.

Joules per Token (J/token) - A normalized energy metric computed by dividing total inference energy by the number of generated tokens. It is used to compare energy efficiency across configurations that may generate responses of different lengths.

Inference Speed - The rate at which the LLM generates output tokens during inference, measured in tokens per second. Higher inference speed indicates faster response generation.

Output Quality - The degree to which the generated response is semantically similar to the reference response. In the study, output quality is measured using BERTScore F1.

BERTScore F1 - A reference-based text evaluation metric that measures semantic similarity between a generated response and a reference response using contextual language embeddings. In the study, it serves as the primary output quality metric.

Resource Utilization - The amount of hardware resources consumed during inference. In the study, resource utilization is measured using CPU utilization percentage, GPU utilization percentage, and memory usage.

CPU Utilization - The percentage of CPU processing capacity used during inference. In the study, CPU utilization is recorded as a supporting resource metric.

GPU Utilization - The percentage of GPU processing capacity used during inference. In the study, GPU utilization is recorded alongside GPU power draw and VRAM usage.

Memory Usage - The amount of system memory used during inference, measured in megabytes. In the study, memory usage is monitored as part of hardware resource utilization.

VRAM Usage - The amount of GPU memory used during inference. In the study, VRAM usage helps determine whether a configuration is feasible on the available GPU hardware.

GPU Layer Offloading - A runtime setting that controls how many model layers are assigned to the GPU during inference. Higher GPU offloading may improve speed but can increase GPU energy consumption and VRAM usage.

CPU Threads - The number of CPU threads allocated to the inference process. In the study, CPU thread allocation is treated as a configurable runtime parameter that may affect speed and resource usage.

Quantization - A model compression technique that reduces the numerical precision of model weights to lower memory and computational requirements. In the study, quantization is evaluated through the Q8_0 and Q4_K_M GGUF variants of Qwen2.5-1.5B-Instruct.

GGUF - A model file format used by llama.cpp for running large language models locally. In the study, Qwen2.5-1.5B-Instruct is evaluated using three GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M.

F16 GGUF - The half-precision GGUF variant used in the study. It represents the highest-precision variant evaluated in the study and reduces memory demand relative to full-precision deployment while preserving higher numerical precision than quantized variants.

Q8_0 GGUF - The 8-bit quantized GGUF variant used in the study. It includes evaluating how 8-bit quantization affects energy consumption, inference speed, output quality, and resource utilization.

Q4_K_M GGUF - The 4-bit quantized GGUF variant used in the study. It is included as the most aggressively compressed model variant among the evaluated GGUF formats, allowing the study to examine whether additional memory and energy reduction can be achieved while preserving acceptable inference speed and output quality.

Qwen2.5-1.5B-Instruct - The base large language model used in the study. It is evaluated through three GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M, all served locally through llama.cpp.

llama.cpp - An open-source C++ inference engine for running GGUF-format large language models on local hardware. In the study, llama.cpp serves as the inference backend for all three experimental conditions.

BOPIS - Bayesian Optimization and Pareto-Based Intelligent Configuration Selection. It is the system proposed in the study for selecting energy-efficient local LLM inference configurations using Bayesian Optimization and Pareto-based trade-off analysis.

Bayesian Optimization - A probabilistic optimization method used to search expensive black-box configuration spaces efficiently. In the study, it is used to identify promising LLM inference configurations with fewer evaluations than exhaustive search.

Black-Box Optimization - An optimization approach applied to systems whose internal input-output relationship cannot be directly derived. In the study, LLM inference is treated as a black-box problem because configuration parameters affect energy, speed, and quality in a nonlinear and hardware-dependent manner.

Gaussian Process (GP) - A probabilistic surrogate model used in Bayesian Optimization. In the study, the GP estimates the expected energy performance of unevaluated configurations and provides uncertainty information to guide search.

Surrogate Model - An approximation model used to estimate the behavior of an expensive objective function. In the study, the Gaussian Process serves as the surrogate model for predicting configuration energy performance during Bayesian Optimization.

Acquisition Function - A function used in Bayesian Optimization to decide which configuration should be evaluated next. In the study, Expected Improvement is used to balance exploration of uncertain configurations and exploitation of promising configurations.

Expected Improvement (EI) - The acquisition function used by BOPIS to select the next configuration for evaluation. It estimates which candidate configuration is expected to improve over the best observed result while considering prediction uncertainty.

Seed Configurations - The initial randomly selected configurations evaluated before Bayesian Optimization begins guided search. In the study, ten seed configurations are used to provide initial observations for the Gaussian Process surrogate model.

Pareto Optimality - A condition in which no objective can be improved without worsening at least one other objective. In the study, Pareto optimality is used to identify balanced trade-off configurations across energy consumption, inference speed, and output quality.

Pareto Front - The set of non-dominated configurations produced through Pareto analysis. In BOPIS, the final recommended configuration is selected from the Pareto front.

Random Search Baseline - An uninformed configuration search method that selects candidate configurations randomly from the same configuration space used by BOPIS. In the study, it serves as the comparison baseline for evaluating the value of Bayesian-guided search.

Two-Way Comparison - The main experimental comparison between the random search baseline, and BOPIS-recommended configuration. This comparison is used to evaluate differences in energy consumption, inference speed, output quality, and resource utilization.

Per-Variant Performance - The descriptive comparison of performance outcomes across the evaluated GGUF precision/quantization variants. In the study, per-variant performance is analyzed across F16, Q8_0, and Q4_K_M in terms of energy consumption, inference speed, BERTScore F1, and resource utilization.

Mean Absolute Error (MAE) - A prediction error metric that measures the average absolute difference between GP-predicted energy values and actual measured energy values. In the study, MAE is used to evaluate Gaussian Process surrogate model accuracy.

Normalized Prediction Error (NPE) - A normalized prediction error metric computed by expressing GP prediction error relative to the mean measured energy. In the study, NPE is used to interpret surrogate prediction reliability as a percentage.

Convergence Behavior - The pattern of improvement observed across Bayesian Optimization iterations. In the study, convergence is evaluated by tracking the best energy value found per iteration.

Improvement per Iteration (ΔE) - The change in the best measured energy value from one optimization iteration to the next. Positive improvement indicates that the search has found a lower-energy configuration.

Sample Efficiency Ratio (SER) - A metric that compares how many iterations BOPIS and random search require to identify their selected configurations. In the study, SER greater

than 1.0 indicates that BOPIS reached its selected configuration in fewer evaluations than random search.

Energy Improvement Ratio (EIR) - A configuration success indicator that measures the percentage reduction in energy consumption of the BOPIS-recommended configuration relative. In the study, EIR greater than 0 indicates energy improvement.

Speed Retention Ratio (SRR) - A configuration success indicator that measures whether the BOPIS-recommended configuration retains acceptable inference speed. In the study, SRR is computed using mean tokens per second.

Quality Retention Ratio (QRR) - A configuration success indicator that measures whether the BOPIS-recommended configuration retains acceptable output quality. In the study, QRR is computed using the mean BERTScore F1.

Wilcoxon Signed-Rank Test - A non-parametric statistical test used to determine there is a significant difference between two related groups. In this study, it is used to compare the random search baseline and the BOPIS-recommended configuration across the same prompt set.

Databricks Dolly 15k - A publicly available instruction-following dataset containing approximately 15,000 human-generated prompt-response pairs across multiple task categories. In the study, it is used as the source of prompts and reference responses for evaluation.

Structured Experiment Papers - Manual recording and verification templates used to document measured performance values during experimentation. In the study, these templates support manual checking, while structured CSV logs serve as the authoritative data source for analysis.

CSV Logs - Structured data files automatically generated by the researchers' Python scripts during experimentation. In the study, CSV logs contain raw and computed values used for descriptive and statistical analysis.

NVIDIA Management Library (NVML) - A driver-level NVIDIA interface used to obtain GPU power, utilization, and memory information. In the study, NVML is accessed through Python ctypes for GPU monitoring.

Linux /proc Filesystem - A virtual filesystem in Linux that provides process and system-level information. In the study, /proc is used to collect CPU utilization, memory usage, and process-related measurements.

Idle Power (P_idle) - The mean GPU power draw recorded before inference begins while no inference process is running. In the study, P_idle is subtracted from sampled GPU power readings to estimate inference-related energy consumption.

Green AI - A principle in AI research that emphasizes reducing computational cost and energy consumption while maintaining acceptable performance. The study follows this principle by treating energy efficiency as a primary optimization objective.

Chapter 2
REVIEW OF LITERATURE AND STUDIES

This chapter presents the body of knowledge related to the research problem. It includes a comprehensive discussion of existing literature, past studies, and related systems relevant to the computing problem being investigated. The purpose of this chapter is to provide a theoretical and technical background of the study, identify gaps in existing research, and justify the need for the proposed research.

Local LLM Inference and The Need for Efficient Configuration Selection

The use of LLM in professional and organizational contexts often involves sensitive and confidential information, Yan et al. (2025) conducted thorough literature review on protecting data privacy in LLM and LLM agent systems. Their findings highlight that routing user inputs through external cloud providers introduces risks of data exposure that are inappropriate in sectors like healthcare, legal services, education, and finance. By guaranteeing that inference takes place within the user's hardware environment and that no data is sent to other servers, local deployment reduces this worry.

Tian et al. (2025) introduced CLONE, a framework for collaborative local-network edge LLM inference, which clearly states that edge LLM deployment must balance latency, energy consumption, and model accuracy because edge and local devices have limited storage, power, and computing capacity compared to data centers. CLONE shows that deploying even moderate sized LLMs on local hardware creates a three-way conflict among response speed, energy consumption, and output quality. This conflict can not be resolved by simply choosing the most powerful configuration because accomplishing so could eventually compromise system stability.

Bast et al. (2024) investigated LLM inference on local hardware, using output quality, inference latency, and energy efficiency as three primary assessment variables. Their research examined numerous locally deployed models and discovered that no model or configuration simultaneously maximizes all three characteristics, showing that local LLM deployment is essentially a multi-objective problem that requires researchers to make conscious trade-off decisions. These studies collectively establish the motivation for BOPIS, because local deployment introduces hardware and energy constraints that cloud environments absorb invisibly, users require a systematic configuration selection approach to determine which local LLM setting is most practical given known constraints.

While these studies effectively demonstrate the importance of efficient local LLM deployment and identify energy consumption, latency, and output quality as the three critical performance dimensions, none of them provide a mechanism for automatically and systematically identifying the inference configuration that achieves the best energy-performance trade-off for a given locally deployed model, leaving the configuration selection problem unsolved.

Energy Consumption and Inference Efficiency in Large Language Models

Energy consumption is becoming a more critical factor in the design and implementation of AI systems. Patterson et al. (2021) conducted a detailed examination of the energy and carbon emissions associated with large-scale machine learning, indicating that hardware efficiency and system design decisions have a significant impact on both operating costs and environmental impact. Their findings revealed that inference workloads, which occur constantly and at scale with each user requests, account for an increasing proportion of overall AI-related energy usage. This is a separate matter from training, which occurs just once per model and so

has a constant cost. In contrast, inference is an ongoing operational expense that grows with each prompt supplied by a user throughout the duration of a deployed system's lifespan.

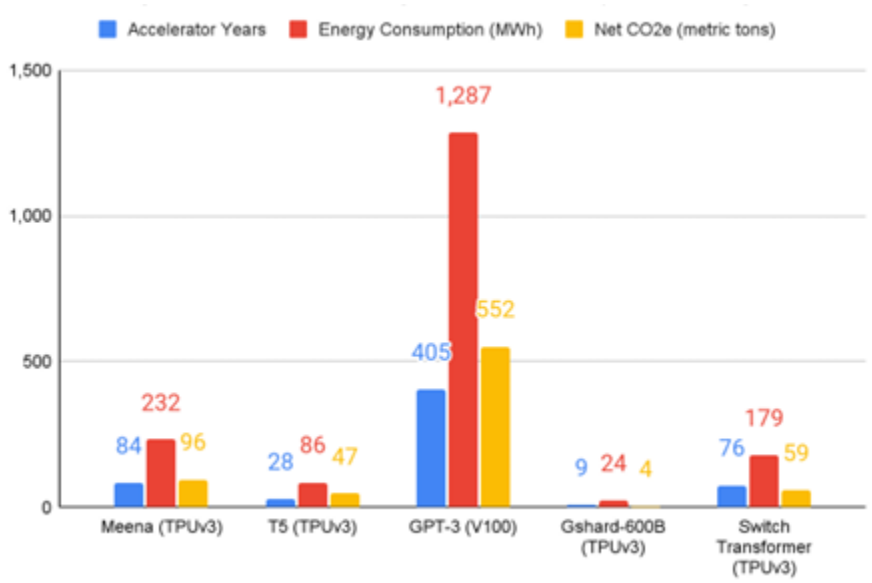


Figure 2.1: Carbon Emissions of Large Neural Network Training by Patterson et al. (2021)

Wilkins et al. (2024) modeled workload-dependent, energy consumption and runtime LLM inference tasks on GPU-CPU systems, resulting in accurate models with R^2 values surpassing 0.96 for each LLM analyzed. The findings showed that Gaussian Process surrogate models trained on LLM inference energy measurements produce reliable predictions from a small number of tested configurations, directly validating the GP-based surrogate modeling approach used in the Bayesian Optimization stage of the study. Furthermore, Hoxha et al. (2025) proposed a deployment-aware framework for carbon- and water-efficient LLM serving that jointly optimized sustainability and performance objectives. Their study emphasized that AI deployment efficiency cannot be evaluated using a single metric alone and that operational sustainability requires balancing multiple competing objectives simultaneously. This supports the need for multi-objective optimization strategies in energy-efficient LLM inference systems.

LLM (# Params)	Energy Model (e_K)			Runtime Model (r_K)		
	R^2	F-statistic	p-value	R^2	F-statistic	p-value
Falcon (7B)	0.964	681.2	2.53e-55	0.962	651.1	1.35e-54
Falcon (40B)	0.972	904.5	1.78e-60	0.976	1073.0	2.74e-63
Llama-2 (7B)	0.973	942.3	3.76e-61	0.972	1032.0	1.19e-62
Llama-2 (13B)	0.972	887.8	3.60e-60	0.972	907.0	1.60e-60
Llama-2 (70B)	0.976	1022.0	6.66e-62	0.980	1230.0	6.23e-65
Mistral (7B)	0.975	997.0	1.70e-61	0.976	1039.0	3.62e-62
Mixtral (8x7B)	0.980	1238.0	4.97e-65	0.992	3139.0	2.23e-80

Figure 2.2: Summary of OLS Regression Results Across Models by Wilkins et al. (2024)

In addition, Schwartz et al. (2020) established the ideas of Green AI, advocating for transparency in computational cost reporting and creating AI systems that consume the least amount of energy while maintaining performance. This work explicitly adopts the Green AI perspective, treating energy consumption as a first-class optimization target and actively identifying lower-energy inference configurations rather than passively measuring energy usage. Husom et al. (2024) emphasized the importance of this framing, demonstrating that combining process-level GPU power monitoring via pynvml with per-token energy normalization produces reliable and comparable inference energy measurements across varying configuration parameters and hardware environments. Their work using the MELODI framework also revealed that energy consumption during LLM inference is primarily driven by response characteristics such as response length and duration rather than prompt complexity – a finding that informs BOPIS configuration space design by confirming that parameters governing the inference process itself, rather than input characteristics, are the primary targets for optimization.

Husom et al. (2024) emphasized the importance of precise energy measurement, demonstrating that combining process-level GPU power monitoring via pynvml with per-token energy normalization produces reliable and comparable inference energy measurements across varying configuration parameters and hardware environments. Their work through the MELODI

framework also revealed that energy consumption during LLM inference is primarily driven by response characteristics such as response length and duration rather than prompt complexity — a finding that informs the BOPIS configuration space design by confirming that parameters governing the inference process itself, rather than input characteristics, are the primary targets for optimization.

Furthermore, Hoxha et al. (2025) proposed a deployment-aware framework for carbon- and water-efficient LLM serving that jointly optimized sustainability and performance objectives, emphasizing that AI deployment efficiency cannot be evaluated using a single metric alone and that operational sustainability requires balancing multiple competing objectives simultaneously. This supports the need for multi-objective optimization strategies in energy-efficient LLM inference systems.

In this study, energy consumption is measured in Joules for total inference energy and normalized as Joules per token to allow fair comparison across configurations that produce outputs of varying lengths. GPU-level energy is estimated through real-time power polling using NVIDIA driver-level monitoring, while CPU utilization and memory-related measurements are collected from operating-system interfaces. This measurement approach supports consistent per-run comparison of energy consumption, inference speed, and resource utilization across all evaluated configurations.

While this body of work establishes the importance of inference energy consumption and validates measurement methodologies, it focuses on profiling and characterizing energy usage rather than optimizing it; none of the studies provide a system that actively searches for and recommends energy-efficient configurations for locally deployed single-machine LLM environments.

Configuration Parameters Affecting Local LLM Performance

The setup settings in BOPIS are based on earlier research that identified which features of the inference process had the most impact on energy consumption, inference speed, and output quality in local deployment contexts. Zhou et al. (2024) conducted a comprehensive assessment of effective LLM inference and discovered three major reasons of inefficiency: high model size, quadratic attention complexity relative to input sequence length, and autoregressive decoding. In local deployment, these sources materialize as practical configuration settings that researchers may change without affecting model weights.

Parameter	Description	Literature Support
Input token length	Longer inputs quadratically increase attention computation, which increases GPU memory and energy demand	Zhou et al. (2024), Vaswani et al. (2023)
Batch size	Controls the number of prompts handled every run, which influences throughput and energy per token.	Stojkovic et al. (2024), DynamoLLM
Precision / Quantization Variant	Controls the deployed GGUF model format used during inference.	Dettmers et al. (2022), Frantar et al. (2022), Wan et al. (2023), Shi et al. (2025)
GPU layers offloaded	Distributes model layers between CPU and GPU, affecting energy demand per component.	CLONE (Tian et al., 2025), Bast et al. (2024)

CPU threads	Controls CPU-side similarity; influences runtime for CPU-bound inference activities.	Bast et al. (2024)

Table 2.1. BOPIS Configuration Parameters and Literature Support

These five parameters were chosen because they can be directly configured on consumer-grade local hardware without requiring model modification, have documented effects on energy consumption and inference performance in existing literature, and represent the practical levers available to practitioners who deploy LLMs locally using tools like llama.cpp. BOPIS considers each unique combination of these parameters to be a candidate configuration, and it uses Bayesian Optimization to efficiently explore the resultant space rather than exhaustively analyzing all possibilities.

Although the reviewed literature individually documents the energy and performance effects of each configuration parameter, no previous study has optimized all five parameters simultaneously as a combined configuration space for energy-efficient local LLM inference — the combination that BOPIS is specifically designed to investigate

.

Bayesian Optimization for LLM Inference Parameter Tuning

Several recent studies have investigated the use of Bayesian Optimization to solve the problem of LLM inference parameter tuning, with each providing evidence for its usefulness as a sample-efficient search approach over expensive black-box objective functions. Jones et al. (1998) introduced Efficient Global Optimization as an early surrogate-based approach for optimizing expensive black-box functions, where each evaluation is costly and the objective function cannot be directly derived. This provides the foundation for using sequential surrogate-guided evaluation in configuration search. Building on this line of work, Snoek et al.

(2012) developed the foundational framework for Bayesian Optimization using Gaussian Process surrogate modeling and Expected Improvement acquisition, demonstrating that this method is especially effective for optimizing costly and unknown objective functions where exhaustive search is computationally inefficient. The recommendation of an initial random exploration phase to start the surrogate model before directed search begins has a direct impact on the sample strategy of this work, which starts the Bayesian Optimization with 10 random configuration evaluations before applying the acquisition function.

Wang et al. (2023) introduced EcoOptiGen, a framework that uses Bayesian Optimization with cost-based reduction to optimize LLM inference hyperparameters such as the number of responses, temperature, and maximum token length, demonstrating that inference hyperparameter optimization provides significant utility gains across GPT-3.5 and GPT-4 models with constrained inference costs. While EcoOptiGen demonstrates the effectiveness of BO-based inference parameter adjustment, its optimization goal is monetary cost rather than energy consumption, and its assessment is restricted to proprietary cloud-based models rather than locally deployed open-source LLMs.

Bayesian Optimization has also been validated in engineering and energy-related optimization systems. Wang et al. (2025) applied Bayesian Optimization to energy management strategies for plug-in hybrid electric vehicles and reported that Bayesian-guided optimization achieved higher efficiency and lower computational cost than conventional optimization approaches. Similarly, Tanim et al. (2024) demonstrated that Gaussian Process-based Bayesian Optimization effectively optimized real-time flood simulation systems while requiring only a limited number of evaluation samples. These findings support the suitability of Bayesian Optimization for systems where evaluations are expensive, nonlinear, and highly configuration-dependent, characteristics that are also present in LLM inference optimization.

Sabbatella et al. (2024) proposed Bayesian Optimization for Instruction Generation (BOInG), demonstrating that Bayesian-guided optimization improved instruction generation quality while minimizing unnecessary evaluations. Likewise, MALBO (Sabbatella, 2025) applied multi-objective Bayesian Optimization to LLM-based multi-agent systems and reported that Bayesian-guided search reduced optimization cost by more than 45% compared to random search while maintaining comparable task performance. These studies collectively demonstrate that Bayesian Optimization is highly effective for optimizing expensive and high-dimensional LLM configuration spaces.

Beyond demonstrating final performance improvement, Bayesian Optimization must also be evaluated based on the reliability and efficiency of its search process. Bayesian Optimization relies on a surrogate model to approximate an expensive black-box objective function and uses this approximation to decide which configuration should be evaluated next. Snoek et al. (2012) and Shahriari et al. (2016) emphasized the Gaussian Process-based Bayesian Optimization is useful for expensive evaluation settings because it models both predicted performance and uncertainty, allowing the research process to balance exploration and exploitation. Therefore, evaluating the reliability of the surrogate model is important because inaccurate predictions may lead the optimization process toward ineffective configurations.

Mean Absolute Error (MAE) is used in this study to measure the average difference between the energy value predicted by the Gaussian Process surrogate model and the actual measured energy value after inference execution. MAE is widely used as a prediction error metric because it directly expresses the average magnitude of model error in the same unit as a measured variable. In the context of BOPIS, MAE helps determine whether the surrogate model is producing energy predictions close to actual hardware measurements. Normalized Prediction Error (NPE) extends this validation by expressing prediction error relative to the measured energy value, making prediction error comparable across configurations with different energy

magnitudes. This is necessary because raw error values can be scale-dependent, while normalized error values allow fairer interpretation across varying output ranges.

The use of prediction-error validation is further supported by Wilkins et al. (2024), who modeled workload-dependent energy consumption and runtime behavior in LLM inference and reported high prediction accuracy for energy and runtime models. Their findings support the idea that energy behavior in LLM inference can be modeled and predicted from measured workload characteristics. In BOPIS, MAE and NPE are therefore used as process-level validation metrics to determine whether the Gaussian Process surrogate model is reliably guiding the search rather than producing uninformed estimates.

Convergence behavior is also necessary for evaluating Bayesian Optimization because the method is expected to improve its selected configurations as more evaluations are performed. Tracking the best energy value found per iteration shows whether the optimization process is moving toward lower-energy configurations over time. The improvement per iteration, represented as ΔE, measured how much additional energy reduction is gained from one iteration to the next. These metrics help identify whether the optimization process is still improving, has started to stabilize, or is merely fluctuating without meaningful progress. This aligns with Bayesian Optimization literature, where sequential improvement and convergence toward better objective values are central indicators of search effectiveness.

Sample efficiency is another important measure because BOPIS is designed to reduce the number of configuration evaluations needed to identify an effective configuration. Bayesian Optimization is commonly justified as a sample-efficient method for expensive black-box problems because it uses previous observations to guide future evaluations. Bergstra and Bengio (2012) established random search as a practical baseline for evaluating more advanced optimization methods because it explores the same search space without model-guided decision-making. In this study, the Sample Efficiency Ratio (SER) compares the number of

iterations required by BOPIS and random search to identify the final Pareto-optimal configuration. A SER greater than 1.0 indicates the BOPIS identified the selected configuration using fewer evaluations than random search, demonstrating the practical value of Bayesian-guided search under the same evaluation budget.

Overall, MAE and NPE validates the predictive reliability of the Gaussian Process surrogate model, convergence and ΔE validate whether the optimization process improves over time, and SER validates whether BOPIS is more sample-efficient than uninformed random search. These metrics ensure that BOPIS is evaluated not only by its final recommended configuration, but also by the reliability and efficiency of the Bayesian Optimization process that produced that recommendation.

LLM Quantization and Numerical Precision

Quantization is among the most consequential configuration decisions for local LLM deployment, directly governing the trade-off between energy efficiency, memory footprint, and output quality. This section surveys the literature supporting BOPIS's inclusion of numerical precision as a core configuration parameter.

Dettmers et al. (2022) demonstrated in LLM.int8() that 8-bit matrix multiplication reduces GPU memory requirements by approximately 50% relative to FP32, with less than 1% degradation in perplexity on standard language modeling benchmarks. The key insight was that only a small fraction of model weights (outlier features) contribute disproportionately to prediction quality, and these can be selectively handled in higher precision while the bulk of the model operates in INT8.

Frantar et al. (2022) extended quantization to 4-bit with GPTQ, a post-training quantization method applicable to models with billions of parameters. Their results showed that

4-bit quantization achieves near-lossless accuracy on most tasks, though with greater sensitivity on tasks requiring precise factual recall. This establishes a quantitative basis for the quality thresholds that BOPIS uses when selecting precision levels for different task types.

Lee et al. (2025) specifically investigated the effect of quantization across different NLP task categories, finding that open-ended generation, summarization, and brainstorming tasks are more robust to quantization than closed-form question answering and information extraction. This task-sensitivity finding is the empirical basis for BOPIS's task-informed prior over precision variants. BOPIS selects a single configuration x* for the deployed workload rather than switching configurations per prompt. The task-informed prior operates on the search's seeding phase and on the per-category quality floor Q_min(task): of the five search parameters, only maximum generation length is a per-request quantity, while precision, GPU layer count, thread count and slot count are inference-server launch parameters that a single running server cannot vary between requests.

Shi et al. (2025) evaluated quantization on CPU-GPU local deployment systems and found that INT8 achieves 1.8x to 2.6x energy reduction per token relative to FP32 on instruction-following benchmarks, with BERTScore F1 degradation remaining below 2% in most cases. This empirical finding establishes the expected magnitude of energy gain from precision reduction that BOPIS targets in its optimization.

Quantization is one of the most widely studied and practically impactful techniques for improving LLM inference efficiency on limited hardware. When model weights and activations are represented during inference, the conventional 32-bit floating point representation is reduced to lower-precision forms like 16-bit floating point or 8-bit integers. This reduction minimizes the amount of floating-point operations needed for each inference step, the model’s memory footprint, and the bandwidth needed to transfer weights through the hardware.

Wan et al. (2023) conducted a comprehensive survey of efficient large language model techniques, categorizing quantization methods into post-training quantization (PTQ), which reduces numerical precision after training without modifying model weights, and quantization-aware training (QAT), which incorporates precision constraints during the training process. While QAT has been shown to improve accuracy retention at low-bit precision by allowing the model to adapt to quantization-induced noise (Krishnamoorthi, 2018; Banner et al., 2019), it requires access to training pipelines, large-scale datasets, and significant computational resources for retraining or fine-tuning.

In contrast, PTQ enables precision reduction directly on pre-trained models and has been demonstrated to maintain strong performance even at INT8 precision (Jacob et al., 2018; Dettmers et al., 2022). This makes PTQ more suitable for deployment-oriented scenarios where models are evaluated at inference time without retraining. Accordingly, PTQ is adopted in the study as it aligns with the objective of optimizing inference-time configurations for locally deployed large language models under practical resource constraints.

Husom et al. (2024) assessed 28 quantized LLMs deployed on an edge device, methodically measuring the impact of quantization on energy efficiency, inference delay, and output correctness, Their findings showed that quantization decreases energy usage and inference time while introducing quality trade-offs that vary with model design and quantization level. Importantly, they discovered that the link between quantization level and quality decrease is non-linear and model-dependent, implying that the best precision level cannot be estimated just by theory and must be empirically evaluated for each model and job. This clearly explains the inclusion of numerical precision as a configuration option in BOPIS's search space, as well as the use of Bayesian Optimization to determine the precision level that best balances energy and quality for the particular model and dataset being evaluated.

In the study, quantization is treated as one of the configurable inference parameters because different model formats produce different trade-offs among energy consumption, inference speed, memory demand, and output quality. BOPIS evaluates three GGUF precision/quantization variants of the same base model, Qwen2.5-1.5B-Instruct: F16, Q8_0, and Q4_K_M. F16 represents the half-precision condition, Q8_0 represents the 8-bit quantized condition, and Q4_K_M represents the 4-bit quantized condition. Using variants of the same base model allows the study to evaluate precision and quantization effects while keeping the model family and architecture constant.

Prior quantization studies support this design because they show that lower-bit inference can reduce memory and computation cost while introducing task-dependent quality trade-offs. Dettmers et al. (2022) demonstrated the practicality of 8-bit quantization for large language models, while Frantar et al. (2022) showed that 4-bit post-training quantization can substantially reduce model size while preserving acceptable performance in many cases. These findings justify the inclusion of both Q8_0 and Q4_K_M in the configuration space. Therefore, BOPIS does not assume that the lowest-bit variant is automatically best; instead, it empirically evaluates the energy, speed, and quality trade-offs of each variant under the same local hardware, dataset, and inference backend.

While quantization research has thoroughly characterized the energy savings and quality trade-offs of individual precision levels in isolation, no previous study has used Bayesian Optimization to automatically discover the optimal precision level as part of a joint multi-parameter configuration search for energy-efficient local LLM inference, which is what BOPIS provides.

Multi-Objective Optimization and Pareto Analysis in LLM Systems

The multi-objective nature of LLM inference optimization, in which energy consumption, latency, and output quality are all affected by the same configuration parameters and frequently conflict with one another, drives the use of Pareto analysis as the configuration evaluation mechanism in this study. Kakolyris et al. (2024) showed that energy efficiency and SLO compliance are competing objectives in LLM inference systems, with some configurations reducing energy at the expense of latency and others maintaining throughput at the expense of high power consumption. Their findings show that no one configuration can concurrently maximize all three dimensions, which requires a Pareto-based strategy that retains the entire range of trade-off solutions rather than combining conflicting objectives into an arbitrary weighted sum.

Tanaka et al. (2026) introduced a multi-objective optimization framework that takes accuracy, latency, memory footprint, and energy consumption into account, identifying Pareto-optimal configurations across 15 models with parameters ranging from 0.5B to 70B and achieving an average 2.8 times improvement in efficiency metrics while maintaining accuracy within 1.2 percent of the baseline. AE-LLM demonstrates the power of Pareto-based multi-objective optimization for LLM efficiency, but its framework focuses on model architecture optimization rather than inference parameter configuration, and it does not use Bayesian Optimization as a search strategy, which are methodological gaps that BOPIS addresses.

Pareto Optimization addresses this challenge by identifying non-dominated solutions, or configurations where no objective can be improved without worsening at least one other objective. Rather than forcing multiple objectives into a single weighted metric, Pareto analysis preserves the trade-off relationship among objectives and allows decision-makers to select configurations according to operational requirements.

Similarly, BAMBO demonstrated that Bayesian Adaptive Multi-objective Optimization effectively generated Pareto-optimal configurations for LLM systems across performance and

efficiency objectives (Chen et al., 2025). The study showed that Pareto-front construction enabled flexible configuration selection while avoiding inefficient dominated solutions. These findings strongly support the methodological design of BOPIS, where Bayesian Optimization is combined with Pareto analysis to identify balanced configurations that minimize energy consumption while maintaining acceptable inference speed and output quality.

Recent sustainability-focused studies also reinforce the relevance of Pareto analysis in AI systems. Hoxha et al. (2025) emphasized that sustainable LLM deployment requires simultaneous optimization of carbon efficiency, water efficiency, and performance objectives rather than isolated optimization of a single metric. Their findings support the use of multi-objective optimization approaches in energy-efficient AI deployment systems.

While Pareto analysis identifies non-dominated configurations, the study still requires decision criteria to determine whether the final selected configuration is practically optimized. Prior studies on local LLM deployment show that energy consumption, inference speed, and output quality are competing dimensions that must be evaluated together rather than independently. Bast et al. (2024) evaluated local LLM deployment using quality, latency, and energy efficiency, showing that no single configuration simultaneously maximizes all performance dimensions. Similarly, Kakolyris et al. (2024) emphasized that energy-efficient LLM inference must preserve performance requirements rather than reducing energy consumption alone. These findings support the use of retention-based success criteria in BOPIS.

In the study, the Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR) are used as operational metrics for interpreting the final BOPIS recommendation. EIR measures whether the optimized configuration reduces energy consumption, aligning with Green AI principles that treat computational efficiency as an evaluation criterion alongside model performance (Schwartz et al., 2020). Since software-based energy and carbon estimation depends on system-level assumptions, energy improvement must

be interpreted using predefined measurement criteria rather than raw differences alone. Lannelongue et al. (2021) showed that computational carbon estimation depends on factors such as processing time, hardware configuration, memory usage, and infrastructure assumptions. Jain (1991) further supports the use of clearly defined metrics and decision criteria in empirical systems performance analysis. These studies support the use of EIR as an operational indicator for energy improvement and justify the adoption of a predefined threshold for identifying substantial improvement.

SRR measures whether inference speed, expressed in tokens per second, is retained after optimization. This prevents the system from selecting a configuration that reduces energy consumption only by making inference impractically slow. QRR measures whether output quality is retained using BERTScore F1, which evaluates semantic similarity between generated and reference responses using contextual embeddings (Zhang et al., 2020). Together, EIR, SRR, and QRR ensure that the final BOPIS recommendation is not selected based on energy reduction alone, but on energy improvement with acceptable speed and semantic quality preservation.

While the reviewed literature validates Pareto analysis as an effective mechanism for dealing with conflicting objectives in LLM systems, existing Pareto-based systems either focus on model architecture optimization rather than inference parameter configuration or operate in cloud and server-side environments rather than local single-machine deployments. Furthermore, existing works rarely formalize configuration success using energy improvement, speed retention, and quality retention criteria. The gap is addressed by BOPIS through a Pareto-based configuration selection process supported by EIR, SRR, and QRR as operational decision indicators.

Output Quality Evaluation using BERTScore F1

Zhang et al. (2020) presented BERTScore, an automated evaluation metric that addresses this limitation by comparing candidate and reference texts with contextual token embeddings from a pretrained language model. BERTScore computes cosine similarity between the contextual representations of each token in the candidate and reference, rather than counting shared words or phrases, and then combines these to get accuracy, recall, and F1 scores. The F1 score, which balances both directions of similarity, has been demonstrated to correlate more strongly with human quality evaluations typical overlap measures across a wide range of text generation tasks.

BERTScore F1 is used as BOPIS' primary output quality indicator for three reasons. First, it assesses semantic similarity rather than surface-level overlap, making it suitable for instruction-following tasks with various viable phrasings of the right answer. Second, it is a reference-based measure that can be calculated automatically against the reference outputs provided by Databricks Dolly 15k without the need for human annotation, making it possible to apply uniformly across all 500 sampled prompts and all three experimental situations. Third, it generates a single normalized score that is directly comparable across configurations with varying inference speeds and energy levels, allowing it to be included as one of three Pareto objectives, along with energy usage and inference speed.

In the study, BERTScore F1 also serves as the basis for the Quality Retention Ratio (QRR), which determines whether the BOPIS-optimized configuration preserves semantic output quality. While BERTScore F1 is a more semantically grounded quality measure than traditional overlap metrics and has been shown to correlate with human judgments, it is still a proxy metric that does not fully capture all dimensions of output quality, such as factual accuracy, coherence, and task-specific correctness, and its scores may differ depending on the pretrained model used as the embedding backbone.

Related Systems and Optimization Frameworks

Several existing systems have explored optimization strategies for improving AI inference efficiency. EcoOptiGen (Wang et al., 2023) applied Bayesian Optimization for cost-efficient LLM inference tuning in cloud environments, while Tanaka et al. (2026) focused on Pareto-based optimization across model architecture and performance objectives. Although these systems demonstrated the effectiveness of Bayesian Optimization and Pareto analysis individually, they did not integrate both techniques into a unified framework specifically targeting local LLM inference energy optimization.

TokenPowerBench (Niu et al., 2025) introduced a benchmarking framework for measuring token-level energy consumption in LLM systems. The framework standardized inference energy measurements and highlighted significant variations in energy efficiency across model configurations. However, the framework focused primarily on profiling and benchmarking rather than active optimization or intelligent configuration selection.

Similarly, Autotuner-inspired random search systems explored automated parameter sampling strategies for inference optimization. While these approaches reduce manual tuning effort, they rely on uninformed exploration and therefore require significantly more evaluations than Bayesian-guided search approaches. Existing literature consistently shows that random search becomes inefficient in large and high-dimensional configuration spaces because it lacks probabilistic learning mechanisms.

Bergstra and Bengio (2012) established random search as a practical, reproducible, and theoretically justified baseline for hyperparameter optimization, demonstrating that random search investigates a broader and more diverse set of important configuration dimensions than grid search, especially when only a few parameters have a significant impact on performance. They claimed that random search should be used as the standard baseline for evaluating more advanced optimization methods since it needs no prior knowledge of the configuration space

and provides an impartial exploration strategy. In BOPIS, random search is used as the secondary comparison condition alongside the default configuration.

Compared to all other systems studied, BOPIS provides a unified offline pre-deployment system that integrates Gaussian Process-based Bayesian Optimization with Pareto-based trade-off analysis for energy-efficient local LLM inference. BOPIS is intended to be executed once before deployment, generating a suggested configuration that is implemented statically for all future inference processes, ensuring that optimization overhead does not affect end-user response time in production. BOPIS also conducts controlled evaluations against random search baselines, utilizing similar datasets, hardware conditions, and statistical validation procedures.

While each of the reviewed systems addresses a relevant aspect of LLM inference optimization — whether through energy profiling, automated search, or multi-objective analysis — none integrate Bayesian Optimization and Pareto analysis into a single unified offline pre-deployment framework that is evaluated against random search baselines under controlled, statistically validated local deployment conditions, as BOPIS does.

Synthesis of the Study

The reviewed literature reveals fundamental findings that collectively validate the design of BOPIS and indicate the specific gap it fills. It also reveals a consistent pattern across five interconnected themes; the recognition of local LLM deployment as an unsolved multi-objective efficiency problem, the empirical grounding of energy consumption as a primary optimization concern, the individual validation of inference configuration parameters as consequential energy controls, and the establishment of Bayesian Optimization as the proper search method for costly configuration spaces, and Pareto analysis is confirmed as the correct framework for dealing with competing performance objectives. Rather than being separate contributions, these bodies of

work complement and support one another, creating the theoretical and methodological basis of BOPIS while exposing a clear and persistent gap that no previous research has addressed.

The most foundational concept in the reviewed literature is that local LLM deployment is essentially a multi-objective challenge, and that existing practice fails to handle it systematically. Yan et al. (2025), Tian et al. (2025), and Bast et al. (2024) arrive at a similar conclusion from different perspectives; local hardware environments create an unavoidable three-way conflict between energy consumption, inference speed, and output quality that no single default configuration can resolve at once. What distinguishes this finding from mere problem identification is its direct application practice. Most organisations deploying LLM locally rely on untuned default configurations without conducting a systematic evaluation of whether those settings are efficient for their specific hardware and workload. This collection of work's strength stems from its empirical basis across numerous hardware environments and model sizes. Their main limitation, however, is that none of these studies goes beyond problem identification. They explicitly state the need for systematic configuration selection, but they do not provide a means for doing so. This absence serves as the primary reason for BOPIS.

Building on this motivation, the energy consumption literature explains why energy should be addressed as a fundamental optimisation objective as well as how it can be reliably assessed – but it does not provide a system to act on those measurements. Patterson et al. (2021) and Schwartz et al. (2020) present an environmental and operational rationale for considering inference energy as a primary issue rather than a secondary reporting metric. Wilkins et al. (2024) and Husom et al. (2024) provide empirical and methodological foundations for this motivation. Wilkins demonstrates that Gaussian Process models achieve R^2 values exceeding 0.96 for LLM energy prediction, directly validating the surrogate modeling approach used in BOPIS. Husom confirms that per-token normalisation through process-level GPU monitoring produces reliable and comparable measurements across configurations. Hoxha et al.

(2025) emphasize that sustained AI deployment requires simultaneous optimisation of several metrics rather than isolated energy savings. Collectively, these studies validate the three-layer energy monitoring infrastructure used in BOPIS, but they highlight the same fundamental limitation; existing profiling systems accurately measure energy without actively discovering better configurations. The change from passive profiling to active optimisation is exactly what BOPIS is intended to accomplish.

The configuration parameter literature then explains what BOPIS should look for and why. Zhou et al. (2024) identify the key causes of LLM inference inefficiency as quadratic attention complexity with respect to input length, high model size, and autoregressive decoding which manifest in local deployment as the five customisable factors investigated by BOPIS. The quantization literature, including Dettmers et al. (2022), Frantar et al. (2022), Lee et al. (2025), Shi et al. (2025), Wan et al. (2023), and Husom et al. (2024), demonstrates that reducing numerical precision can lower memory demand and computational cost while introducing task-dependent quality trade-offs. In this study, this concept is operationalized through three GGUF precision/quantization variants of the same base model, Qwen2.5-1.5B-Instruct: F16, Q8_0, and Q4_K_M. These variants allow BOPIS to evaluate how half precision, 8-bit quantization, and 4-bit quantization affect energy consumption, inference speed, output quality, and resource utilization under the same hardware environment, prompt dataset, and inference backend. Since the quality impact of quantization is nonlinear and model-dependent, empirical evaluation is necessary rather than relying on theoretical assumptions alone. Tian et al. (2025) and Bast et al. (2024) confirm GPU layer offloading and CPU thread allocation as system-level characteristics that influence hardware workload distribution and energy consumption. Taken together, these studies show that each of the five BOPIS configuration parameters has measurable implications on energy and performance outcomes. Their common disadvantage, however, is that each parameter is evaluated independently. No previous work has investigated

how these factors interact when optimized as a combined configuration space, which is what BOPIS is doing.

Given the difficulty and expense of assessing configurations in this combined space, the Bayesian Optimization literature explains why BO is not only a practical but also conceptually acceptable search method. Jones et al. (1998) and Snoek et al. (2012) provide a theoretical support for surrogate-based optimization of expensive black-box functions, demonstrating that a Gaussian Process surrogate combined with the expected improvement acquisition function allows for efficient search by learning from each evaluation and directing subsequent ones toward the most promising part of the configuration space. EcoOptiGen (Wang et al., 2023), MALBO (Sabbatella, 2025), BOInG (Sabbatella et al., 2024), and cross-domain validations by Wang et al. (2025) and Tanim et al. (2024) all confirms that BO consistently outperforms uninformed search strategies, with MALBO reporting a 45% reduction in optimization cost when compared to random search. The surrogate validation framework established in this chapter emphasizes that BO should be evaluated not only on its final configuration recommendation, but also on the reliability of its surrogate predictions, as measured by MAE and NPE, and the efficiency of its convergence across iterations. Existing BO applications to LLM inference have a persistent limitation in that they target monetary cost on cloud-based proprietary models rather than energy consumption on locally deployed open-source LLMs, and none combine BO with Pareto analysis for multi-objective configuration selection in local deployment scenarios. BOPIS advances immediately beyond this barrier.

Finally, the Pareto analysis literature reveals that after desirable configurations have been identified via BO, a principled multi-objective evaluation framework is required to choose between them. Kakolyris et al. (2024) show empirically that energy, speed, and quality are competing objectives in LLM inference systems; decreasing energy frequently affects latency or quality, implying that no single configuration prevails across all dimensions. Tanaka et al. (2026)

and BAMBO (Chen et al., 2025) demonstrate that Pareto-based multi-objective optimization identifies balanced and efficient configurations across LLM systems, with Tanaka reporting a 2.8 times increase in efficiency metrics while maintaining accuracy within 1.2% of baseline. Hoxha et al. (2025) emphasize that successful AI deployment requires a multi-objective approach to competing efficiency factors. The consistent limitation across this literature is that existing Pareto frameworks rely on model architecture rather than inference parameters, or they operate in cloud and server-side environments rather than local single-machine deployments, neither of which addresses the configuration selection problem that BOPIS solves.

These five themes together reveal a clear and definite gap. Energy profiling techniques can measure inference energy but do not actively optimize it. Dynamic scheduling solutions improve energy efficiency at the infrastructure level but are often designed for server-side or cloud environments. Pareto-based approaches address multi-objective trade-offs, but many focus on model architecture or deployment-level optimization rather than local inference configuration. Bayesian Optimization methods efficiently search costly configuration spaces, but existing applications often optimize monetary cost or task performance in proprietary or cloud-based models rather than energy consumption in locally deployed LLM inference. No reviewed study combines Bayesian Optimization and Pareto Analysis into a unified offline pre-deployment system that jointly evaluates runtime configuration parameters and GGUF precision/quantization variants, including F16, Q8_0, and Q4_K_M, for locally deployed LLM inference. BOPIS addresses this gap through a statistically validated two-way comparison between random search baseline, and BOPIS-recommended configuration under controlled experimental conditions.

CHAPTER 3

METHODOLOGY

This chapter describes the procedures and methods used to address the research problem and achieve the objectives of the study. It explains how the research was conducted, how data were collected and generated, how the BOPIS system was developed, and how the results will be analyzed and evaluated. The chapter is organized into nine sections: research design, sources of data, sampling data, system architecture, research instruments, data generation and gathering procedure, ethical considerations, data analysis, and statistical treatment.

Research Design

The study employs a quantitative experimental research design to evaluate the effectiveness of BOPIS in selecting energy-efficient configurations for locally deployed Large Language Model (LLM) inference under controlled conditions. The design is experimental because the study compares measurable performance outcomes across different configuration selection strategies using the same hardware environment, basemodel, prompt dataset, inference backend, and measurement procedures.

The research design is organized into five procedural stages. These stages describe how the experiment is conducted to answer the Statement of the Problem and evaluate the proposed system.

The first stage defines the configuration search space. This includes input token length, batch size, precision/quantization variant, GPU layer offloading, and CPU thread allocation. The study uses the same base model, Qwen2.5-1.5B-Instruct, across three GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M. These variants are included to

determine how half precision, 8-bit quantization, and 4-bit quantization affect energy consumption, inference speed, output quality, and resource utilization.

The second stage evaluates two comparative conditions: the random search baseline and the BOPIS-optimized configuration. The random search baseline represents an uninformed stochastic search method that selects configurations from the same search space without model-guided decision-making. The BOPIS-optimized configuration is generated through Bayesian Optimization followed by Pareto-based multi-objective selection. The unoptimized default configuration is measured as a reference point only and is not included in the Wilcoxon signed-rank comparison. It serves as the denominator for EIR, SRR, and QRR

The third stage collects performance data from the same standardized prompt dataset across all two conditions. A fixed stratified sample from Databricks Dolly 15k is used to ensure fair and consistent comparison. The dependent variables include energy consumption, inference speed, output quality, and resource utilization. Energy consumption is measured in Joules and Joules per token, inference speed is measured in tokens per second, output quality is measured using BERTScore F1, and resource utilization is measured through CPU utilization, GPU utilization, and memory usage. Per-variant performance is also summarized across F16, Q8_0, and Q4_K_M to examine how each GGUF variant affects the measured outcomes.

The fourth stage evaluates the reliability and efficiency of the Bayesian Optimization process used by BOPIS. Gaussian Process surrogate model accuracy is assessed using Mean Absolute Error (MAE) and Normalized Prediction Error (NPE) between GP-predicted energy values and actual measured energy values. Convergence behavior is evaluated using the best energy value found per optimization iteration and the improvement per iteration (ΔE). Sample efficiency is evaluated using the Sample Efficiency Ratio (SER), which compares how efficiently

BOPIS and random search identify their selected configurations under the same evaluation budget.

The fifth stage analyzes and validates the results through descriptive and inferential statistical treatment. Descriptive statistics are used to summarize the performance of each condition using mean, standard deviation, minimum, and maximum values. The Wilcoxon signed-rank test is used to determine whether statistically significant differences exist between the random search baseline and the BOPIS-optimized configuration. The final BOPIS recommendation is further interpreted using Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR) to determine whether energy reduction is achieved while maintaining acceptable inference speed and output quality.

Through this research design, the study determines whether the BOPIS-optimized configuration is energy-efficient compared with random search baseline, while also validating whether the Bayesian Optimization process is reliable, convergent, and sample-efficient.

Sources of Data

The study draws on three categories of data sources: a standardized prompt dataset, hardware performance measurements, and configuration search outputs generated during the experimental runs.

The primary prompt dataset used between both comparison conditions and the default reference is Databricks Dolly 15k, a publicly available instruction-following dataset containing approximately 15,000 human-generated prompts across multiple task categories: open question answering, closed question answering, information extraction, summarization, classification, creative writing, brainstorming, and general-purpose instruction following. Dolly 15k was selected because it provides diverse prompt types representative of common LLM usage,

includes reference outputs that enable proxy-based quality evaluation, and is publicly accessible under its applicable license terms. The dataset is sourced from the official Hugging Face repository at https://huggingface.co/datasets/databricks/databricks-dolly-15k.

The second source of data is direct hardware measurement during local LLM inference. Performance metrics, including energy consumption, inference speed, CPU utilization, GPU utilization, and memory usage, are collected during each experimental run using software-based monitoring tools described in the Research Instruments section. These measurements are generated by running the same base model, Qwen2.5-1.5B-Instruct, across the evaluated GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M. All variants are tested on the study's single-machine CPU-GPU setup under the random search baseline, and BOPIS-optimized configuration.

The third source of data is the configuration evaluation logs generated during the BOPIS Bayesian Optimization process and random search baseline. Each evaluated configuration produces a record of its parameter values, including input token length, batch size, precision/quantization variant, GPU layer offloading, and CPU thread allocation, along with measured energy consumption, inference speed, output quality, and resource utilization. For BOPIS, the logs also include Gaussian Process surrogate predictions, actual measured energy values, prediction error values used for MAE and NPE, best energy value per iteration, improvement per iteration (ΔE), acquisition-related outputs, and Pareto front status. For the random search baseline, the logs record the evaluated configurations and the iteration at which the selected configuration is identified. These data are used to evaluate final configuration performance, per-variant performance, convergence behavior, and sample efficiency through the Sample Efficiency Ratio (SER).

Sampling Data

A stratified random sampling size approach is used to select a subset of prompts from the Databricks Dolly 15k dataset. From the full dataset of approximately 15,000 prompts, a final evaluation sample of 500 prompts is extracted to ensure computational feasibility while maintaining representation across task categories.

Stratification is performed based on the task categories of the dataset, including open question answering, closed question answering, information extraction, summarization, classification, creative writing, brainstorming, and general-purpose instruction following. The number of prompts selected from each category is proportionally allocated based on the category distribution of the dataset. Within each category, prompts are randomly selected without replacement to avoid duplicate entries in the sample.

Additional filtering is applied before sampling. Prompts with missing reference outputs are excluded because output quality is evaluated using BERTScore F1, which requires reference responses. Prompts that exceed the maximum token length supported by the tested configurations are also excluded to ensure consistent processing across all experimental conditions.

The resulting 500-prompt dataset is fixed prior to experimentation and is used consistently between random search baseline, and BOPIS-optimized configuration. This ensures that observed performance differences are attributable to the configuration selection strategy rather than variation in the prompt dataset.

A smaller stratified subset of 50 prompts is selected from the 500-prompt evaluation dataset for intermediate evaluations during the Bayesian Optimization and random search configuration search. This subset serves as a proxy evaluation set that reduces computational cost while preserving proportional representation across task categories. Using the full

500-prompt dataset for every candidate configuration would substantially increase the number of inference runs per optimization iteration; therefore, the 50-prompt subset makes repeated configuration evaluation feasible on the study's single-machine hardware.

The 50-prompt subset is used only during intermediate configuration search and is not used as the sole basis for final performance reporting. After the BOPIS and random search configurations are selected, the final comparative evaluation is conducted using the full 500-prompt dataset under the same measurement procedures. This separation between search-time evaluation and final evaluation helps ensure that the final reported energy consumption, inference speed, output quality, and resource utilization reflect performance across the broader evaluation sample.

System Architecture

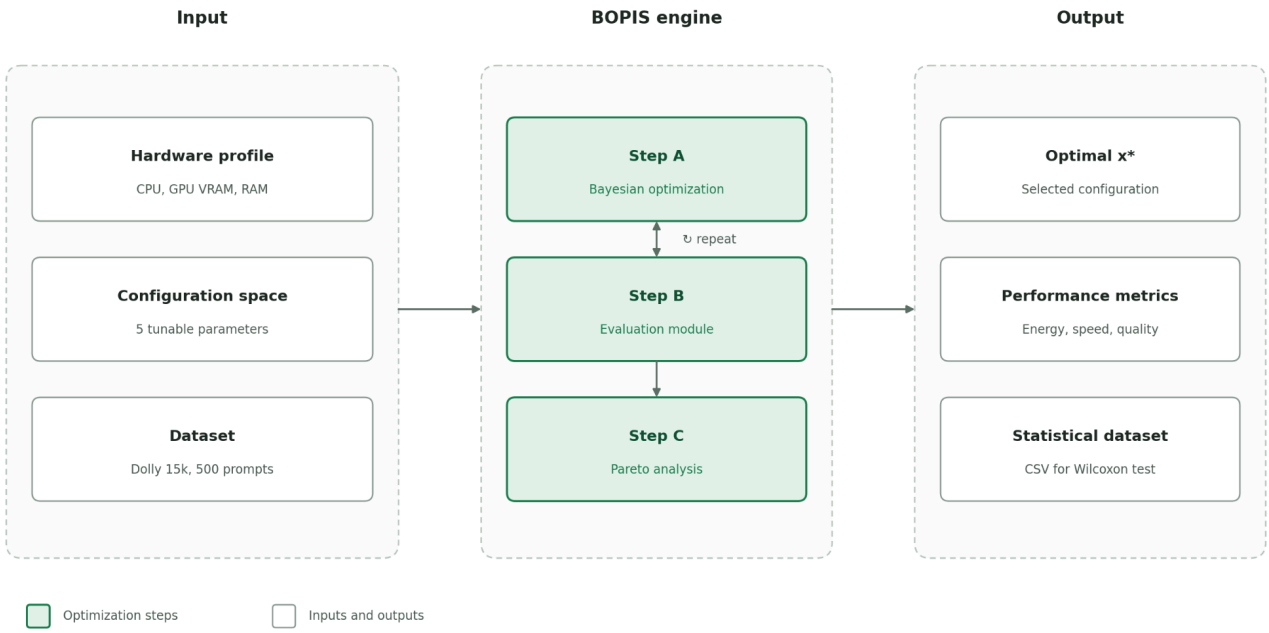


Figure 3.1 System Architecture

The BOPIS system architecture represents the high-level data flow and component interaction required to optimize local large language model inference, illustrating how initial hardware constraints are systematically transformed into a validated deployment recommendation. The system operates sequentially across three primary structural blocks: the Input layer, the BOPIS engine, and the Output layer.

The Input layer establishes the foundational boundaries and raw materials for the experiment. It captures the hardware profile, specifically CPU, GPU VRAM, and RAM, to define the physical limits of the system. It also defines the configuration space, which consists of 5 tunable parameters, and provides the dataset containing 500 prompts from Databricks Dolly 15k. These inputs feed directly into the central BOPIS engine, which executes a repeating cycle of three operations: Step A performs Bayesian optimization to intelligently navigate the search space, Step B serves as the evaluation module to test configurations, and Step C applies Pareto analysis to weigh the performance trade-offs. The pipeline concludes at the Output layer, which consolidates the engine's findings into the optimal selected configuration (x^*), the corresponding performance metrics for energy, speed, and quality, and a structured statistical dataset formatted as a CSV for the Wilcoxon signed-rank test.

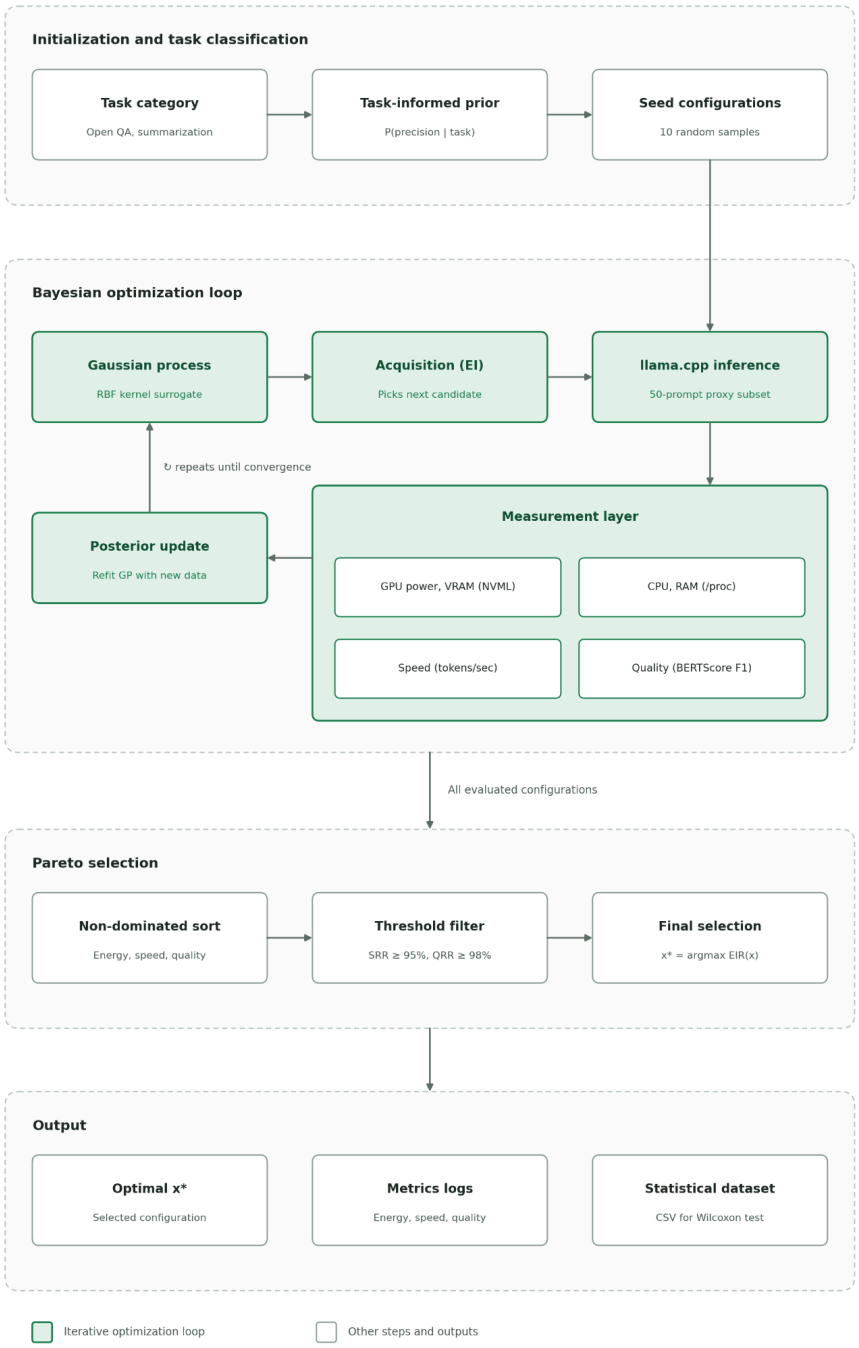


Figure 3.2 Model Architecture

Figure 3.2 shows the internal, step-by-step mechanics of the BOPIS engine. The engine begins with initialization and task classification, in which the task category of the prompt, such as open QA or summarization, is identified. This category defines a task-informed prior,

P(precision | task). The prior is then used to guide the sampling of 10 random seed configurations from the feasible configuration space to start the search and provide initial observation.

The engine then enters the Bayesian optimization loop, which repeats for a fixed number of N total iterations. A Gaussian process with an RBF kernel serves as the surrogate model of the objective space. The acquisition function, Expected Improvement (EI), evaluates this surrogate and selects the next candidate configuration. The candidate is evaluated throug llama.cpp inference on a 50-prompt proxy subset of the 500-prompt dataset, which keeps each iteration fast; the full 500-prompt evaluation dataset is reserved exclusively for the final validation of the selected configuration. During inference, the measurement layer collects hardware and performance data: GPU power and VRAM through NVML, CPU and RAM through the Linux/proc filesystem, inference speed in token per second, and output quality as BERTScore F1. These measurements trigger a posterior update, in which the Gaussian process is refitted with the new data.

The final phase is Pareto selection over all evaluated configurations. Configurations are evaluated in a three-dimensional objective space of energy, speed, and quality, and a non-dominated sort identifies the Pareto front. A threshold filter then discards any configuration that falls below a Speed Retention Ratio (SRR) of 95% or a Quality Retention Ratio (QRR) of 98% each measured relative to the unoptimized default configuration. From the remaining configurations, the engine selects x* = argmax EIR(x), where EIR is the Energy Improvement Ratio, defined as EIR = [(E_default - E(x)) | E_default] x 100%, which isolates the most energy-efficient configuration. This produces the optimal x*, the metrics logs, and the statistical dataset.

Stage	Component	Method	Output
0	Hardware Profiling	Rule-based constraint table	Feasible configuration space
1	Task Classification	Rule-based probability table	Prior P(precision | task)
2	Bayesian Optimization	GP surrogate + EI acquisition	Candidate configurations
3	Gaussian Process	Kernel regression + posterior update	μ(x), σ(x) per configuration
4	Pareto Analysis	Non-dominated sorting	Pareto front + x*
5	Validation	EIR, SRR, QRR, MAE, NPE, SER	Final recommended configuration

Table 3.1. BOPIS System Pipeline Table Overview

The Input Layer provides the resources and parameters required for inference optimization and evaluation. It includes the hardware environment specifications, the llama.cpp inference engine, the configuration search space parameters, and the Databricks Dolly 15K dataset used for benchmark prompts and reference outputs. These inputs are shared consistently across all experimental conditions to ensure fair comparison.

Inside the BOPIS Optimization Engine, the Configuration Space Definition module establishes the discrete search space explored during optimization. Inside the BOPIS Optimization Engine, the Configuration Space Definition module establishes the discrete search space explored during optimization. The configuration parameters include input token length (128, 256, 512, 1024), batch size (1, 2, 4, 8), precision/quantization variant precision/quantization variant (F16, Q8_0, Q4_K_M) , GPU layer offloading (0, 14, 28, All), and CPU thread allocation (2, 4, 8). These parameters are evaluated consistently between random search baseline, and BOPIS optimization condition. The three precision/quantization variants

represent deployment formats of the same base model, Qwen2.5-1.5B-Instruct, allowing the study to evaluate precision and quantization effects while keeping the model architecture constant. These parameters are evaluated consistently between random search, and BOPIS optimization conditions.

Prior to optimization, the hardware environment is profiled to define the feasible configuration space. This is a deterministic rule-based step in which each hardware specification maps directly to a set of permissible parameter values. No configurations outside this feasible space are evaluated during any experimental condition. Table H1 summarizes the hardware constraint rules applied to GPU VRAM, system RAM, and CPU core count.

Hardware Spec	**Condition**	**Constrained Parameter**	**Permitted Values**	**Rule ID**
GPU VRAM	< 4 GB	Precision/Quantization Variant	Q8_0, Q4_K_M	HW-P1
GPU VRAM	4 GB – < 8 GB	Precision/Quantization Variant	F16, Q8_0, Q4_K_M	HW-P2
GPU VRAM	≥ 8 GB	Precision/Quantization Variant	F16, Q8_0, Q4_K_M	HW-P3
GPU VRAM	< 4 GB	GPU Layers Offloaded	0, 14	HW-G1
GPU VRAM	4 GB – < 8 GB	GPU Layers Offloaded	0, 14, 28	HW-G2
GPU VRAM	≥ 8 GB	GPU Layers Offloaded	0, 14, 28, All	HW-G3
System RAM	< 8 GB	Batch Size	1	HW-B1
System RAM	8 GB – < 16 GB	Batch Size	1, 2	HW-B2
System RAM	≥ 16 GB	Batch Size	1, 2, 4, 8	HW-B3

CPU Cores	< 4	CPU Threads	2	HW-C1
CPU Cores	4 – 7	CPU Threads	2, 4	HW-C2
CPU Cores	≥ 8	CPU Threads	2, 4, 8	HW-C3

Table H1. Hardware-Aware Configuration Constraint Rules

Task Classification Module

Before optimization begins, BOPIS infers the task category of the deployed workload in order to select the appropriate prior distribution over precision variants and the corresponding per-category quality floor Q_min(task). Two implementations were developed. The first is a rule-based classifier comprising 42 lexical and structural rules implemented using the Python standard library only, which allows it to run inside the measurement path without introducing additional dependencies. The second is a supervised multinomial Naive Bayes classifier trained on the Databricks Dolly 15k category labels using a 70/15/15 train/validation/test split with a fixed random seed.

On the held-out test split, the supervised classifier achieves 69.7% exact eight-way accuracy against the rule baseline's 48.2%, 78.6% against 63.6% when general and open question answering are merged, and 79.2% against 67.1% when evaluated at the level of quality-sensitivity tier, which is the granularity the prior actually consumes.

Three qualifications apply to this comparison. The dominant residual confusion is between general and open question answering, accounting for 200 of 683 test errors; Dolly distinguishes these categories by a property of the reference answer rather than of the prompt, so the confusion is intrinsic to the labelling scheme. The rule baseline continues to outperform the supervised classifier on closed question answering (89.1% against 81.2%) and open question answering (78.5% against 77.0%), suggesting a hybrid arrangement as future work. Finally, the rule baseline was hand-tuned using corpus-level feedback and has therefore

effectively observed the test distribution, so the reported margin should be read as a lower bound on the improvement.

The feasible configuration space is the Cartesian product of all permitted values across the five configuration parameters after hardware constraint filtering, formally defined as:

$$X_{feasible} = T \times B \times P(HW) \times G(HW) \times C(HW)$$

where T = {128, 256, 512, 1024} represents input token lengths, B(HW) represents batch sizes permitted by available system memory,P(HW) includes three GGUF variants of Qwen2.5-1.5B-Instruct: F16, Q8_0, and Q4_K_M, precision/quantization variants permitted by the available hardware and llama.cpp backend, G(HW) represents GPU layer offloading options permitted by GPU VRAM, and C(HW) represents CPU thread allocation values permitted by CPU core count. In this study, P(HW) includes three GGUF variants of Qwen2.5-1.5B-Instruct: F16, Q8_0, and Q4_K_M. All Bayesian Optimization and random search operations are restricted to this feasible configuration space.

Task classification introduces probabilistic prior knowledge into the Bayesian Optimization search. Based on Lee et al. (2025), different NLP task types exhibit different sensitivity to quantization-induced quality degradation. Open-ended generation, summarization, and brainstorming tasks are more robust to low-bit precision, while closed-form question answering and information extraction are more sensitive. This empirical finding is formalized as a prior probability distribution P(precision | task_type) that weights the initial random sampling phase of the BO engine, as shown in Table T1.

Based on prior studies showing that task types differ in sensitivity to quantization-induced degradation, the study uses a researcher-defined task-informed prior distribution during the initial seed configuration phase. This prior does not represent exact probabilities reported by the cited studies; rather, it operationalizes their general finding that some tasks are more sensitive to low-bit quantization than others.

Task Type	Quality Sensitivity	P(F16)	P(INT8)	P(Q4_K_M)	Min. Precision	Literature Basis
Open QA	**Low**	0.53	0.235	0.235	P(F16)	Lee et al. (2025)
Closed QA	**High**	0.69	0.233	0.077	P(F16)	Lee et al. (2025)
Summarization	**Low**	0.47	0.265	0.265	P(F16)	Lee et al. (2025)
Classification	**Low**	0.39	0.305	0.305	P(Q8_0)	Lee et al. (2025)
Creative Writing	**Low**	0.53	0.235	0.235	P(F16)	Lee et al. (2025)
Brainstorming	**Low**	0.39	0.305	0.305	P(Q8_0)	Lee et al. (2025)
Info Extraction	**High**	0.69	0.233	0.077	P(F16)	Lee et al. (2025)
General Instr.	**Medium**	0.56	0.264	0.176	P(F16)	Shi et al. (2025)

Table T1. Researcher-Defined Task-Informed Prior Distribution for GGUF Precision/Quantization Variant Selection

For a dataset containing mixed task types, as in Databricks Dolly 15k, the effective dataset-level prior is computed as a weighted average across all task categories:

$$P(\text{precision}) = \Sigma_t \left[P(\text{precision} \mid \text{task}_t) \times P(\text{task}_t) \right]$$

where $P(\text{task}_t)$ is the proportion of the 500-prompt evaluation sample belonging to task type t, derived from the stratified sampling distribution. This produces a dataset-level prior that concentrates initial evaluations on precision levels most appropriate for the overall task composition of the dataset, improving the efficiency of the early exploration phase.

The Bayesian Optimization Engine serves as the primary intelligent search mechanism of BOPIS. The engine uses a Gaussian Process (GP) surrogate model to approximate the relationship between inference configurations and observed performance outcomes. The Expected Improvement acquisition function selects candidate configurations by balancing

exploration of uncertain regions and exploitation of configurations predicted to yield favorable results. The surrogate model is iteratively updated after each inference evaluation using newly collected measurements from the Evaluation Module.

The BO engine executes the following six-step procedure across all optimization iterations:

Step 1 Initialize: Draw 10 seed configurations from X_feasible using the task-informed variant prior P(p | task). Evaluate each on the 50-prompt BO subset. Record $\{x_i, E(x_i), S(x_i), Q(x_i)\}$ for i = 1…10.

Step 2 Fit GP Surrogate: Fit a Gaussian Process to all observed $\{x_i, E(x_i)\}$ pairs. The GP provides predicted mean μ(x) and uncertainty σ(x) for any unevaluated configuration.

Step 3 Acquire Next Config: Select the next configuration x_{t+1} by maximizing the Expected Improvement (EI) acquisition function over X_feasible.

Step 4 Evaluate: Run llama.cpp inference under x_{t+1} on the 50-prompt subset. Record $E(x_{t+1}), S(x_{t+1}), Q(x_{t+1})$.

Step 5 Update: Add the new observation to the dataset. Refit the GP posterior. Return to Step 3.

Step 6 — Terminate: After N total iterations (10 random seed + N−10 BO-guided). Apply Pareto analysis to all evaluated configurations.

The Expected Improvement (EI) acquisition function selects the next configuration by balancing exploration of uncertain regions with exploitation of known high-performing areas:

$$EI(x) = (\mu(x) - f(x^+)) \times \Phi(Z) + \sigma(x) \times \varphi(Z)$$

$$Z = (\mu(x) - f(x^+)) / \sigma(x)$$

where μ(x) is the GP-predicted mean energy for configuration x. σ(x) is the GP uncertainty (standard deviation). $f(x^+)$ is the best energy value observed so far. Φ(Z) is the standard normal cumulative distribution function. φ(Z) is the standard normal probability density function. The configuration with the highest EI across all unevaluated members of X_feasible is selected as the next configuration to evaluate.

The Gaussian Process (GP) surrogate model approximates the unknown energy function E(x) from a small number of observations, providing both a predicted mean and an uncertainty estimate for every configuration in the feasible space. The GP prior is parameterized by a mean function m(x) and the Radial Basis Function (RBF) kernel:

$$k(x, x') = \sigma^{f2} \times exp(-||x - x'||^2 / (2l^2))$$

where σ^{f2} is the output variance controlling the amplitude of energy variation across the configuration space, and l is the length scale controlling how quickly the function varies. Both are hyperparameters fitted by maximizing the log marginal likelihood.

After observing n configuration-energy pairs $D_n = \{(x_1, E_1), \ldots, (x_n, E_n)\}$, the GP posterior provides updated predictions for any unevaluated configuration x:

$$\mu_n(x) = m(x) + k(x, X_n) \times [K(X_n, X_n) + \sigma^{e2} I]^{-1} \times (y_n - m(X_n))$$

$$\sigma_n^2(x) = k(x,x) - k(x, X_n) \times [K(X_n, X_n) + \sigma^{e2} I]^{-1} \times k(X_n, x)$$

where $\mu_n(x)$ is the posterior predicted mean energy. $\sigma_n^2(x)$ is the posterior variance (uncertainty). $K(X_n, X_n)$ is the n×n kernel matrix of observed configurations. $k(x, X_n)$ is the 1×n cross-covariance vector. $y_n = [E_1, \ldots, E_n]^T$ is the vector of observed energy values. σ^{e2} is the noise variance. I is the n×n identity matrix.

The GP hyperparameters $\{\sigma^{f2}, l, \sigma^{e2}\}$ are refitted after each new observation by maximizing the log marginal likelihood:

$$\log p(y_n \mid X_n, \theta) = -\tfrac{1}{2}\, y_n^T [K + \sigma^{e2} I]^{-1} y_n - \tfrac{1}{2} \log|K + \sigma^{e2} I| - (n/2) \log(2\pi)$$

where $\theta = \{\sigma_f^2, l, \sigma_e^2\}$ are optimized using L-BFGS-B implemented through Python standard library math operations, keeping the surrogate model calibrated throughout the search.

The Evaluation Module executes LLM inference using llama.cpp subprocess calls under the candidate configuration provided by the optimization engine. Each evaluated configuration produces inference outputs and execution logs which are forwarded to the Performance Measurement Layer for monitoring and analysis.

The Performance Measurement Layer operates passively across the entire optimization and benchmarking pipeline. This layer records energy consumption, inference speed, output quality, and resource utilization metrics for every evaluated configuration. GPU power draw, utilization, and VRAM consumption are collected directly through the NVIDIA Management Library (NVML) accessed via Python ctypes. CPU utilization, RAM usage, and thread activity are retrieved from the Linux /proc filesystem. Inference speed in tokens per second is extracted from llama.cpp execution logs, while output quality is evaluated using BERTScore F1 against Databricks Dolly 15K reference outputs. All measurements are logged into structured CSV datasets using researcher-developed scripts without reliance on third-party monitoring frameworks.

The Pareto Analysis Module performs multi-objective evaluation on all configurations evaluated during the Bayesian Optimization search. The module identifies Pareto-optimal solutions across the competing objectives of minimizing energy consumption, maximizing inference speed, and maximizing output quality. From the resulting Pareto front, BOPIS selects the final recommended configuration based on energy-efficiency priority while maintaining acceptable performance thresholds.

A configuration x is said to dominate configuration x', written $x \prec x'$, if and only if it is at least as good on all three objectives and strictly better on at least one:

$$E(x) \leq E(x') \text{ AND } S(x) \geq S(x') \text{ AND } Q(x) \geq Q(x'), \text{ with at least one strict inequality}$$

The Pareto front PF is the complete set of non-dominated configurations across all evaluated candidates:

$$PF = \{ x \in X_evaluated : \forall\ x' \in X_evaluated,\ x' \text{ does not dominate } x \}$$

The final recommended configuration x* is selected from the Pareto front as the configuration that achieves the greatest energy reduction while satisfying both performance constraints:

$$x^* = \operatorname{argmax} EIR(x)$$

$$\text{subject to: } x \in PF \ \text{AND}\ SRR(x) \geq 95\% \ \text{AND}\ QRR(x) \geq 98\%$$

where EIR(x) = [(E_default − E(x)) / E_default] × 100% is the Energy Improvement Ratio. SRR(x) = [S(x) / S_default] × 100% is the Speed Retention Ratio. QRR(x) = [Q(x) / Q_default] × 100% is the Quality Retention Ratio. x* is the configuration that maximizes energy savings while meeting both performance constraints simultaneously.

The quality of the Pareto front as a whole is measured using the Hypervolume (HV) indicator, which quantifies the volume of objective space dominated by the front as the reference worst-case point:

$$HV(PF, r) = \lambda(\{ q \in \mathbb{R}^3 : \exists\ x \in PF,\ f(x) \preccurlyeq q \preccurlyeq r \})$$

where λ denotes the Lebesgue measure (three-dimensional volume). r = (E_default, −S_default, −Q_default) is the reference worst-case point. A larger HV indicates a Pareto front that covers a wider and better trade-off space.

To validate optimization effectiveness, the architecture includes a comparison condition: the random search baseline. These conditions bypass the Bayesian Optimization Engine but still execute through the same evaluation and measurement pipeline to ensure experimental consistency. All three conditions generate benchmark results and structured statistical analysis datasets.

The Output Layer consolidates the experimental outputs of the system. These outputs include benchmark comparison results, statistical analysis datasets, Pareto front solutions, and the final optimal configuration identified by BOPIS. The statistical analysis pipeline applies descriptive statistics, to determine whether statistically significant differences exist between random search baseline, and BOPIS optimization model.

Table F1 summarizes the complete BOPIS pipeline, mapping each processing step to its inputs, methods, output metrics, and the recording table where measurements are logged.

Pipeline Step	**Input**	**Process**	**Output Metric(s)**	**Table**
Hardware Profiling	GPU VRAM, CPU cores, RAM	Apply HW constraint rules	X_feasible, |X_feasible|	Table H1
Task Classification	Prompt task type	Look up precision prior	P(precision|task)	Table T1
BO Init (10 seeds)	X_feasible + P(precision)	Sample using task-informed prior	Seed configs $\{x_1 \ldots x_{10}\}$	Table A.7
GP Fit	Observed $\{x_i, E_i\}$	Fit RBF kernel + optimize θ	μ(x), σ(x) for all x	Table A.7
EI Acquisition	μ(x), σ(x), $f(x^+)$	Maximize EI over X_feasible	Next config x_{t+1}	Table A.7
Inference Evaluation	x_{t+1}, 50-prompt subset	Run llama.cpp, collect metrics	E, S, Q, CPU%, GPU%, RAM	Table A.7
GP Update	New (x_{t+1}, E_{t+1})	Refit GP posterior	Updated μ, σ, ΔE	Table A.7

GP Validation	All $\{\mu(x_i), E_i, \sigma(x_i)\}$	Compute MAE, NPE, R^2, UCR	Surrogate reliability metrics	Table M1
Pareto Analysis	All evaluated configs	Non-dominated sorting	Pareto front PF, HV	Table A.11
x* Selection	PF + SRR/QRR thresholds	argmax EIR subject to constraints	Final config x*	Table A.10
3-Way Validation	Full 500-prompt set	Run Unopt, RS, BOPIS x*	EIR, SRR, QRR, all raw metrics	Tables A.1–A.6
Statistical Analysis	Per-prompt metric vectors	Wilcoxon signed-rank	p-values, significance pairs	Table A.6

Table F1. BOPIS Full Pipeline Flow — Input, Process, Output Metrics, and Data Tables

Table 3.2 summarizes the configuration parameters and their candidate values explored by the BOPIS system.

Parameter	**Values Explored**	**Default Value**	**Notes**
Maximum generation length (n_predict)	128, 256, 512, 1024	1024	Per-request cap on generated tokens; context window fixed at ctx_size = 2048 for all runs
Batch size	1, 2, 4, 8	1	Prompts per inference run
Precision/Quantization Variant	F16, Q8_0, Q4_K_M	F16	GGUF deployment variant

GPU layers offloaded	0, 14, 28, All	All	Layers routed to GPU vs CPU
CPU threads	2, 4, 8	System default	Parallelism for CPU-bound ops

Table 3.2. BOPIS Configuration Search Space

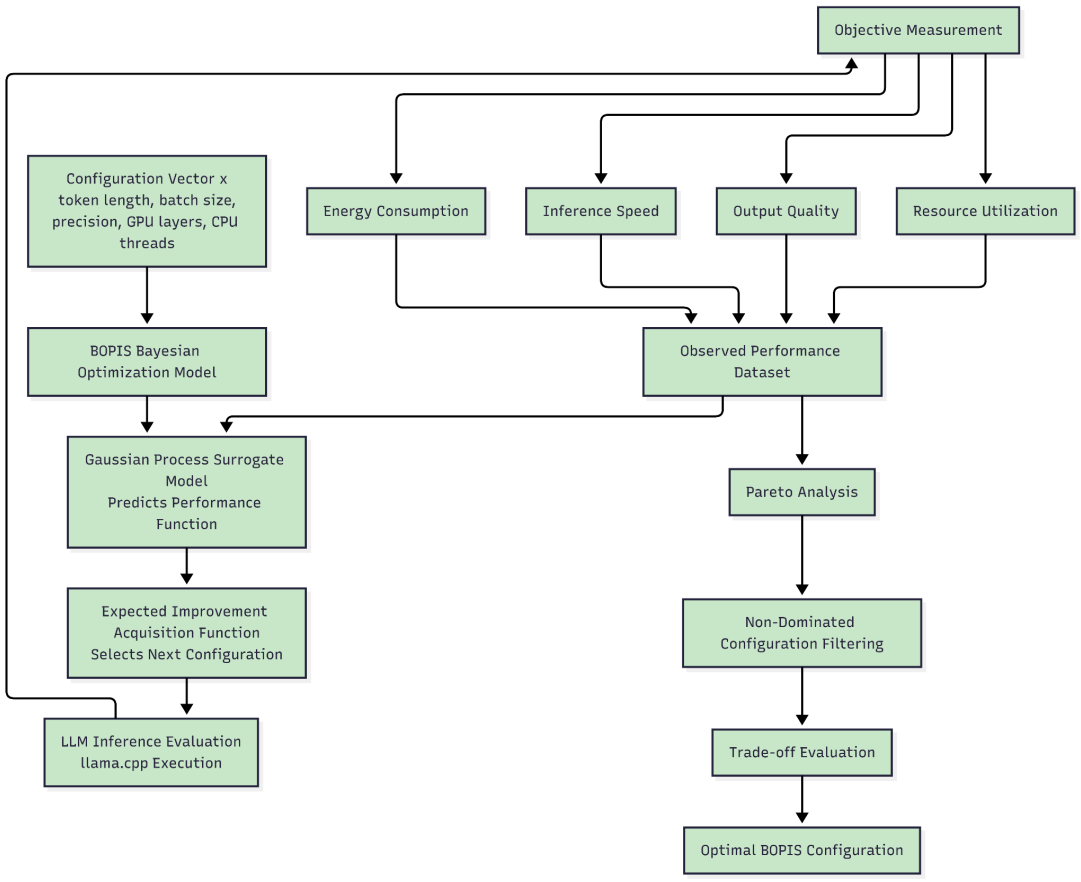


Figure 3.3 BOPIS Optimization Model

Figure 3.3 presents the theoretical optimization model of BOPIS. The model formalizes how Bayesian Optimization and Pareto-based selection are used to identify energy-efficient inference configurations for locally deployed Large Language Models (LLMs). Unlike the system

architecture, which focuses on component interaction and data flow, the BOPIS Optimization Model describes the computational decision-making process used to evaluate, optimize, and select inference configurations.

The optimization process begins with the definition of the inference configuration vector:

$$x = (t, b, p, g, c)$$

where t represents input token length, b represents batch size, p represents the GGUF precision/quantization variant, g represents GPU layer offloading configuration, and c represents CPU thread allocation. The value of where p is selected from {F16, Q8_0, Q4_K_M}, representing the three evaluated deployment variants of the same base model.

The study treats LLM inference optimization as a constrained multi-objective optimization problem. The objective function of BOPIS is defined as:

$$f(x) = \{ E(x),\ -S(x),\ -Q(x) \}$$

where E(x) = energy consumption, S(x) = inference speed, Q(x) = output quality. Since optimization procedures are conventionally formulated as minimization problems, inference speed and output quality are represented as negative terms to indicate maximization objectives.

The primary optimization goal of BOPIS is to minimize energy consumption while maintaining acceptable inference speed and output quality thresholds. This is formally represented as:

$$x^* = \operatorname{argmin} E(x)$$

$$\text{subject to: } S(x) \geq S_{min}(H) \text{ and } Q(x) \geq Q_{min}(task)$$

where x represents the configuration search space. S_min represents the minimum acceptable inference speed, defined by Speed Retention Ratio SRR ≥ 95%. Q_min(task) represents the minimum acceptable output quality threshold per task type, defined by Quality Retention Ratio QRR ≥ 98%.

Bayesian Optimization is used as the primary search mechanism of the model. A Gaussian Process (GP) surrogate model approximates the relationship between inference configurations and observed performance outcomes. The surrogate model is iteratively updated after each evaluation using newly observed measurements collected from LLM inference execution. The Expected Improvement (EI) acquisition function balances exploration of uncertain regions in the search space and exploitation of configurations predicted to produce favorable results.

After candidate configurations are evaluated, Pareto analysis is applied to identify non-dominated solutions across the competing objectives of energy consumption, inference speed, and output quality. A configuration is considered Pareto-optimal if no other configuration improves one objective without degrading at least one other objective. This condition is formally defined as: $f_i(x) \leq f_i(x^*)$ for all objectives i, and $f_j(x) < f_j(x^*)$ for at least one objective j.

The resulting Pareto front represents the set of optimal trade-off configurations identified by BOPIS. From this set, the final recommended configuration is selected based on energy-efficiency priority while preserving acceptable performance thresholds for inference speed and output quality.

Research Instrument

The study uses structured experiment papers, automated CSV logs, software-based monitoring scripts, task classification module alongside the monitoring scripts, and a controlled local inference environment as research instruments for collecting and verifying performance data during local Large Language Model (LLM) inference. These instruments are applied consistently across the two experimental conditions: the random search baseline, and the BOPIS-recommended configuration. This ensures that all measured values are collected using the same procedures and can be compared fairly.

Hardware and Software Environment

The study is conducted on a single workstation running Ubuntu 22.04.3 LTS under Windows Subsystem for Linux 2 (WSL2) on a Windows 11 host. All inference execution, monitoring, and logging operations are performed within the WSL2 Ubuntu environment. The hardware and software environment used in the study is summarized in Table 3.3.

Component	**Specification**
GPU	11th Gen Intel Core i5-1135G7 @ 2.40 GHz
CPU	NVIDIA GeForce MX330
RAM	16 GB, approximately 15.78 GiB available
GPU VRAM	2 GB GDDR5
Operating System	Windows 11
Host OS	Windows 11
CUDA Toolkit	12.x
NVIDIA Driver	528.96
LLM Inference Engine	llama.cpp (build b3447, CUDA backend enabled)
LLM Model	Qwen2.5-0.5B-Instruct
Python Version	Python 3.11.6
Power Monitoring	NVML where supported; Windows system monitoring and resource-allocation estimation

Table 3.3. Hardware and Software Environment Specifications

The NVIDIA MX330 is the binding hardware constraint that determines the feasible batch size, GPU layer offloading, and precision/quantization variant settings. Since the study evaluates F16, Q8_0, and Q4_K_M GGUF variants of the same base model, each variant is tested under the same local hardware and measurement pipeline.

Structured Experiment Paper and CSV Logs

The structured experiment papers serve as manual recording and verification templates for documenting measured values during each experimental run. These templates record the dependent variables of the study, including total energy consumption in Joules, normalized energy per generated token, inference throughput in tokens per second, BERTScore F1, CPU utilization, GPU utilization, and memory usage.

At the same time, all measured values are written to structured CSV log files by the researchers' Python logging scripts. The CSV logs serve as the authoritative data source for statistical analysis, while the experiment papers support manual checking, documentation, and verification of recorded values. Any values recorded in the experiment papers are cross-checked against the corresponding CSV log entries before analysis. The complete experiment paper templates are provided in Appendix B.

Inference Engine

The LLM is executed using llama.cpp, an open-source C++ inference engine for running GGUF-format large language models on local hardware. llama.cpp is invoked through Python's subprocess module as an external command-line process. This allows the researchers to run the model consistently across all experimental conditions while capturing execution logs for each inference call.

The llama.cpp execution logs provide wall-clock inference duration and token count information, including prompt tokens and generated tokens. These values are parsed by the researchers' Python scripts and used to compute inference throughput in tokens per second.

Hardware Profiling

Hardware monitoring is performed using the NVIDIA Management Library (NVML) and the Linux /proc filesystem. GPU power draw is measured through NVML using Python's ctypes

interface, allowing the monitoring script to access NVIDIA driver-level power readings without relying on third-party Python wrappers. GPU power readings are collected in milliwatts at fixed 100-millisecond intervals throughout each inference call and are integrated over the inference duration to compute total GPU energy consumption in Joules.

GPU utilization percentage and VRAM usage are also collected through NVML at the same 100-millisecond sampling interval. CPU utilization, RAM usage, thread activity, and process-level memory information are collected from the Linux /proc filesystem. Specifically, CPU time counters are read from /proc/stat, memory availability is read from /proc/meminfo, and process-level thread count and memory footprint are read from /proc/[pid]/status.

Before experimentation, the research machine is profiled to identify the hardware specifications that constrain the BOPIS configuration space. CPU model, core count, and thread count are obtained from /proc/cpuinfo. GPU model, total VRAM, and CUDA-related information are obtained through NVML and nvidia-smi. These hardware details are used to define which configuration values can be tested safely on the available single-machine setup.

Output Quality Measurement

Output quality is evaluated using BERTScore F1, which measures semantic similarity between generated responses and reference outputs. Generated responses from the locally deployed LLM are compared with the corresponding reference outputs from the Databricks Dolly 15k dataset. BERTScore F1 is used because it evaluates contextual semantic similarity rather than exact word overlap, making it suitable for instruction-following tasks where multiple valid phrasings may express the same answer.

Development and Experiment Tools

The following tools support the development, execution, monitoring, and documentation of the experimental setup. These tools support the research process but are distinguished from the dependent-variable measurements themselves.

Tool	Category	Purpose
Visual Studio Code	Code Editor	Primary development environment for writing Python scripts, experiment runners, and data logging utilities.
Python 3.x	Programming Language	Core language for all scripting. Only Python standard library modules are used (subprocess, ctypes, math, csv, json, time, os).
llama.cpp	LLM Inference Engine	Open-source C++ inference engine for running quantized LLMs locally. Invoked via subprocess; provides execution logs including token counts and wall-clock timing.
Git / GitHub	Version Control	Source control for experiment scripts, configuration records, and manuscript files.
nvidia-smi	GPU Query CLI	NVIDIA System Management Interface. Command-line tool included with NVIDIA drivers; used to query GPU model, VRAM capacity, and CUDA capability during hardware profiling.
Microsoft Excel	Spreadsheet	Manual transcription and review of experiment paper values; cross-checking computed metrics against raw log entries.

Table 3.4. Development and Experiment Tools

The complete structured experiment paper templates for performance comparison and configuration evaluation are included in Appendix B. These templates support documentation and manual verification, while the structured CSV logs remain the primary data source used for statistical analysis.

Data Generation/Gathering Procedure

Data collection proceeds through five stages executed in the following order. All monitoring, logging, and metric extraction are performed using direct operating system and driver-level interfaces, specifically the Linux /proc filesystem and the NVIDIA NVML library

accessed through Python ctypes. Python scripts developed by the researchers are used to execute inference calls, collect measurements, parse logs, and generate structured CSV files.

Stage 1: Environment Setup and Instrument Verification

The single-machine CPU-GPU hardware environment is configured and validated before data collection begins. llama.cpp is installed and verified using the three selected Qwen2.5-1.5B-Instruct GGUF variants: F16, Q8_0, and Q4_K_M. The researchers' Python scripts for invoking llama.cpp, accessing NVML through ctypes, parsing /proc filesystem entries, and writing CSV logs are tested using a warm-up run of ten consecutive inference calls. The results of the warm-up run are discarded. This process is conducted to stabilize GPU clock behavior, memory allocation, and runtime conditions before formal measurement begins.

A baseline idle power measurement is then recorded. With no inference process running, the NVML ctypes interface samples GPU power draw every 100 milliseconds for 60 consecutive seconds. The mean of these samples is stored as P_idle and is used during energy computation to distinguish inference-related energy consumption from background idle power draw.

The 500-prompt stratified sample from Databricks Dolly 15k is prepared and saved to disk together with its corresponding reference outputs. A smaller 50-prompt stratified subset is selected from the 500-prompt sample and saved separately for intermediate Bayesian Optimization and random search evaluations.

Stage 2: Unoptimized Default Reference Measurement

The LLM is executed under the unoptimized default configuration across all 500 sampled prompts. Each prompt is submitted individually to llama.cpp through the researchers' Python execution script. During each inference call, GPU power draw is sampled through NVML at

100-millisecond intervals and stored with timestamps. After each call, llama.cpp execution logs are parsed to extract wall-clock inference duration and generated token count. CPU utilization, RAM usage, GPU utilization, and VRAM usage are collected through the same /proc and NVML-based monitoring pipeline.

All raw measurements, generated responses, token counts, timing values, and hardware utilization readings are written to structured CSV log files. After all 500 prompts are completed, output quality is computed by comparing generated responses with the corresponding Databricks Dolly 15k reference outputs using BERTScore F1.

Stage 3: BOPIS Optimization Run

The BOPIS optimization process is initialized using the defined configuration search space. Ten randomly selected seed configurations are evaluated first to provide the initial observed data required to fit the Gaussian Process surrogate model. After this initialization phase, the Expected Improvement acquisition function is used to select succeeding candidate configurations for evaluation.

For each Bayesian Optimization iteration, the selected candidate configuration is executed through llama.cpp using the 50-prompt stratified subset. Each candidate configuration includes input token length, batch size, precision/quantization variant, GPU layer offloading, and CPU thread allocation. For each evaluated configuration, energy consumption, inference speed, output quality, and resource utilization are collected using the same monitoring and logging procedures used in the baseline stage. The observed result is then used to update the Gaussian Process surrogate model.

Across optimization iterations, the system records GP-predicted energy values, actual measured energy values, prediction errors, best energy value found per iteration, and

improvement per iteration. These values are later used to compute MAE, NPE, convergence behavior, and ΔE.

After the fixed number of Bayesian Optimization iterations is completed, all observed configurations are evaluated through Pareto analysis. Non-dominated sorting is performed to identify configurations where no objective can be improved without worsening another objective. The final BOPIS-recommended configuration is selected from the Pareto front according to the study's success criteria. The selected BOPIS configuration is then validated using the full 500-prompt dataset under the same measurement pipeline used in Stage 2.

Stage 4: Random Search Baseline

A random search baseline is executed for comparison against BOPIS. Candidate configurations are sampled uniformly at random from the same configuration search space used by BOPIS, including the same input token length values, batch size values, precision/quantization variants, GPU layer offloading options, and CPU thread allocation values. The number of random search evaluations is matched to the number of Bayesian Optimization evaluations to ensure an equal evaluation budget.

Each random search configuration is evaluated using the same 50-prompt stratified subset and the same monitoring pipeline used in the BOPIS optimization run. After all random search configurations are evaluated, the best-performing random search configuration is selected based on the same objective criteria. The selected random search configuration is then validated using the full 500-prompt dataset under the same measurement pipeline used for the unoptimized default and BOPIS-optimized configurations.

Stage 5: Data Consolidation

All CSV log files from the unoptimized default configuration, BOPIS optimization run, random search baseline, and final 500-prompt validation runs are consolidated into a structured

dataset using the researchers' Python scripts. The final comparative dataset contains per-prompt records for the two comparison conditions and the unoptimized default reference.

For each condition, the consolidated dataset includes energy consumption, Joules per token, inference speed, BERTScore F1, CPU utilization, GPU utilization, and memory usage. Performance is also summarized by the precision/quantization variant to compare the observed behavior of F16, Q8_0, and Q4_K_M across the evaluated configurations. The Bayesian Optimization and random search process logs are retained separately for computing process-level metrics, including MAE, NPE, convergence behavior, ΔE, and SER. Final configuration success indicators, including EIR, SRR, and QRR, are computed using the full 500-prompt evaluation results. Raw CSV log values are cross-verified against the structured experiment paper entries before the dataset is passed to descriptive and inferential statistical analysis.

Ethical Considerations

This study does not involve human participants, personally identifiable information, or private enterprise data. The Databricks Dolly 15k dataset used in the study is a publicly available instruction-following dataset released under the Creative Commons Attribution-ShareAlike 3.0 Unported (CC BY-SA 3.0) license. The dataset is used in accordance with its license terms, and the dataset source is properly cited throughout the study.

The large language model deployed in the study is Qwen2.5-1.5B-Instruct, served through llama.cpp using three GGUF precision/quantization variants: F16, Q8_0, and Q4_K_M. These variants are used to evaluate how different local deployment formats affect energy consumption, inference speed, output quality, and resource utilization under the same base model and inference backend. The model and its variants are used in accordance with the applicable Apache 2.0 license. No proprietary models are used, and no model weights are redistributed beyond what is permitted by the applicable model license. The llama.cpp inference

engine and all supporting open-source tools are used in accordance with their respective licenses.

All data generated during the study, including energy logs, inference speed records, output quality scores, hardware utilization measurements, and configuration records, are produced through the researchers' own experimental setup. These generated records do not contain human participant data or personally identifiable information. Data files are stored on the research hardware, and access is restricted to the research team.

The study adheres to research integrity principles by applying the same hardware environment, prompt dataset, experimental conditions, and measurement procedures across all compared configurations. Results are reported as measured, including cases where the BOPIS-recommended configuration does not outperform the random search baseline on a given metric. No data are selectively omitted, modified, or misrepresented to favor a particular outcome.

All sources cited in the study are properly attributed following APA 7th edition citation standards. Published figures and materials from prior works are used only with proper citation and attribution to their original authors and sources.

Data Analysis (Procedure and Treatment)

Data analysis proceeds through four main components: descriptive analysis, energy computation, Bayesian Optimization process validation, and final configuration evaluation. Inferential statistical analysis is then applied to determine whether the observed differences among the three evaluated configurations are statistically significant.

In the descriptive phase, performance data for all dependent variables are summarized per condition using mean, standard deviation, minimum, and maximum values computed across the 500-prompt evaluation runs. Energy consumption is reported both as total Joules and as

normalized Joules per token (J/token). Inference speed is reported as mean tokens per second (tokens/sec). Output quality is reported using mean BERTScore F1. Resource utilization is reported through mean CPU utilization, GPU utilization, and memory usage. These descriptive statistics provide the overall performance profile of each configuration condition.

In addition to the two-way comparison between random search baseline, and BOPIS-optimized configuration, performance is also summarized by the GGUF precision/quantization variant. The F16, Q8_0, and Q4_K_M variants are compared descriptively in terms of energy consumption, inference speed, BERTScore F1, and resource utilization to determine how model deployment format affects local inference performance.

Energy consumption is computed from GPU power readings collected through NVML during each inference call. Since idle GPU power is measured before experimentation, inference-related energy is computed by subtracting the recorded idle power from the sampled GPU power readings before integrating power over inference duration. The energy per inference run is computed as:

$$E = \Sigma [(P_t - P_{idle}) \times \Delta t]$$

where P_t is the sampled GPU power reading at time t, P_{idle} is the mean idle GPU power recorded during the baseline idle measurement, and Δt is the sampling interval. Since power is recorded in milliwatts, the values are converted to watts before computing Joules. Energy per token is then computed as:

$$J/token = E / N_{generated}$$

The discrete GPU used in the study does not expose a power sensor; both NVML power-query interfaces return NOT_SUPPORTED on this device. Energy is therefore reported as a Mode C estimate derived from vendor power budgets and measured utilization rather than

from direct sensor readings, and every reported energy value is labelled as estimated. Absolute Joules-per-token figures are consequently not claimed as calibrated measurements. The Energy Improvement Ratio remains interpretable because it is a ratio of two estimates produced by the same estimator on the same host, so any systematic multiplicative bias in the estimator cancels; the ratio, not the absolute magnitude, is the quantity the study's conclusions rest on.

In addition to measuring the dependent variables across both comparison conditions, the study evaluates the internal performance of the BOPIS Bayesian Optimization process through process-level metrics. These metrics address whether the Bayesian Optimization engine functions reliably and efficiently, independent of the final configuration outcome, and support the Validation stage (Stage 5) of the BOPIS pipeline described in System Architecture. These process-level metrics are computed from the Bayesian Optimization and random search logs generated during the 50-prompt intermediate configuration search.

First, Gaussian Process surrogate accuracy is measured using Mean Absolute Error (MAE) and Normalized Prediction Error (NPE). Surrogate accuracy is additionally validated by leave-one-out cross-validation over the evaluated configurations, reporting the leave-one-out coefficient of determination (LOO R^2) and leave-one-out NPE. A LOO R^2 of at least 0.85 is required for the surrogate to be treated as reliable. The Gaussian Process predictive variance used for this assessment includes the learned noise term σ_n^2; the noise term is mandatory in σ_{pred}, since omitting it would report predictive intervals narrower than the measurement process can support. MAE quantifies the average absolute difference between the energy value predicted by the GP surrogate before inference execution and the actual measured energy value after inference execution:

$$MAE_{GP} = (1/N) * \Sigma \, | \, \mu(x_i) - E_{measured}(x_i) \, |$$

where N is the total number of Bayesian Optimization evaluation iterations, μ(x_i) is the GP-predicted energy for configuration i before inference is executed, and E_measured(x_i) is the actual energy recorded after inference execution.

NPE normalizes the MAE relative to the mean measured energy:

$$NPE = (MAE_GP / E_mean) \times 100\%$$

where E_mean is the mean measured energy across evaluated Bayesian Optimization configurations. NPE allows prediction error to be interpreted as a percentage of measured energy, making the error easier to compare across configurations with different energy magnitudes. An NPE below 10% is treated as a researcher-defined reliability threshold for this study and is interpreted alongside the observed MAE values and convergence behavior.

Second, convergence behavior is assessed by computing the best energy value found up to and including each iteration k:

$$E^*_k = \min\{ E_measured(x_i) : i \leq k \}$$

where E*_k is the lowest measured energy value observed across all configurations evaluated up to and including iteration k. This produces a non-increasing curve over the optimization process. The per-iteration improvement is computed as:

$$\Delta E_k = E_\{k-1\} - E_k$$

where ΔE_k represents the additional energy reduction gained at iteration k. Positive ΔE values indicate that the optimization process has discovered a lower-energy configuration compared with previous iterations, while repeated zero or near-zero values indicate convergence or stabilization.

Third, sample efficiency is measured by identifying the iteration at which each method first reaches its final best configuration:

$$SER = k^*_{RandomSearch} / k^*_{BOPIS}$$

where k_{BOPIS} *is the earliest iteration at which the configuration selected as the final BOPIS recommendation was first evaluated, and* $k_{RandomSearch}$ is the corresponding iteration for the random search baseline. Both methods use the same evaluation budget. An SER greater than 1.0 indicates that BOPIS identified its selected configuration in fewer evaluations than random search, providing evidence of sample efficiency.

After evaluating the reliability and efficiency of the Bayesian Optimization process, the final BOPIS-recommended configuration is assessed using three configuration success criteria: Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR). These ratios are computed using the full 500-prompt evaluation results and are measured relative to the unoptimized default configuration.

The Energy Improvement Ratio (EIR) measures the percentage reduction in energy consumption achieved by the BOPIS-optimized configuration relative to the unoptimized default configuration:

$$EIR = [(E_{default} - E_{BOPIS}) / E_{default}] \times 100\%$$

where $E_{default}$ is the mean energy consumption, measured in Joules, of the unoptimized default configuration across the 500-prompt evaluation set, and E_{BOPIS} is the mean energy consumption of the BOPIS-recommended configuration on the same set. A configuration is considered energy-improved if $EIR > 0$. An $EIR \geq 15\%$ is adopted as a researcher-defined operational threshold for substantial improvement. This threshold is used to avoid treating small changes in software-based energy estimates as meaningful improvements, since computational energy and carbon estimation depends on system-level factors such as

processing time, hardware configuration, memory usage, and infrastructure assumptions (Lannelongue et al., 2021). Following the principles of empirical computer systems performance analysis, the threshold is treated as a predefined decision boundary for the study rather than a universal scientific constant (Jain, 1991).

The Speed Retention Ratio (SRR) verifies whether optimization preserves acceptable inference throughput:

$$SRR = (S_BOPIS / S_default) \times 100\%$$

where S_BOPIS and S_default are the mean inference speeds, measured in tokens per second, of the BOPIS-optimized and unoptimized default configurations, respectively. An SRR ≥ 95% is required for the configuration to be classified as speed-retaining.

The Quality Retention Ratio (QRR) verifies whether output quality is preserved:

$$QRR = (Q_BOPIS / Q_default) \times 100\%$$

where Q_BOPIS and Q_default are the mean BERTScore F1 scores of the BOPIS-optimized and unoptimized default configurations, respectively. A QRR ≥ 98% is required for the configuration to be classified as quality-retaining.

The formal BOPIS success criterion is defined as:

$$x^* \text{ is BOPIS-optimal if } EIR > 0,\ SRR \geq 95\%,\ \text{and } QRR \geq 98\%.$$

These criteria are treated as predefined practical acceptability thresholds. If any of the three criteria are not met, the configuration may still be discussed as an energy-performance trade-off, but it will not be classified as successfully optimized under the formal BOPIS success criteria.

In the inferential phase, the Wilcoxon signed-rank test is applied separately to each dependent variable to determine whether statistically significant differences exist between the

two evaluated configurations: the random search baseline and the BOPIS-optimized configuration. Since the same 500-prompt dataset is evaluated under both conditions, the Wilcoxon signed rank-test is appropriate for comparing paired samples without assuming normality. The null hypothesis states that there is no statistically significant difference between the two configurations for a given dependent variable. The alternative hypothesis states that there is a statistically significant difference between them. A significance level of $\alpha = 0.05$ is used for all statistical tests. If the computed p-value is less than 0.05, the null hypothesis is rejected.

All statistical analysis is conducted using the researchers' Python scripts. Visualizations, including grouped bar charts, energy-performance trade-off plots, and Pareto front visualizations, are generated from the structured CSV datasets to support interpretation of the results.

Statistical Treatment

The statistical treatment applied in the study is intended to determine whether statistically significant differences exist between random search baseline, and BOPIS-optimized configuration across the measured dependent variables. It also summarizes the descriptive and process-level metrics used to interpret the final configuration performance and the reliability of the Bayesian Optimization process.

The Wilcoxon signed-rank test will be used as the primary inferential statistical test. It will be applied separately to each dependent variable. The unoptimized default configuration is measured as a reference point only and is not included in the Wilcoxon signed-rank comparison. It serves as the denominator for EIR, SRR, and QRR. The null hypothesis states that there is no statistically significant difference between the two configurations. The alternative hypothesis states that the configurations differ significantly.

The null hypothesis states that there is no statistically significant difference among the three configurations for a given dependent variable. The alternative hypothesis states that at least one configuration differs significantly from the others. A significance level of $\alpha = 0.05$ will be used for all statistical tests. If the computed p-value is less than 0.05, the null hypothesis will be rejected.

Descriptive statistics, including mean, standard deviation, minimum value, and maximum value, will also be computed for all dependent variables to summarize the performance of each configuration. Per-variant performance across F16, Q8_0, and Q4_K_M GGUF precision/quantization variants will be analyzed descriptively in terms of energy consumption, inference speed, BERTScore F1, and resource utilization.

BERTScore F1 will be used as the primary output quality metric because it evaluates semantic similarity between generated responses and reference outputs using contextual embeddings. This makes it suitable for evaluating LLM-generated responses where multiple valid phrasings may express the same meaning. BERTScore baseline rescaling is enabled, and absolute F1 deltas are reported alongside QRR. Without rescaling, BERTScore F1 values occupy a narrow high band, and a retention criterion of QRR ≥ 98% would be nearly impossible to fail, carrying no discriminative information.

In addition to statistical comparison, the study will compute configuration success indicators and Bayesian Optimization process metrics. Energy Improvement Ratio (EIR), Speed Retention Ratio (SRR), and Quality Retention Ratio (QRR) will be used to determine whether the BOPIS-recommended configuration achieves energy improvement while retaining acceptable inference speed and output quality. Mean Absolute Error (MAE), Normalized Prediction Error (NPE), convergence behavior, improvement per iteration (ΔE), and Sample

Efficiency Ratio (SER) will be used to evaluate the reliability and efficiency of the Bayesian Optimization process.

Table 3.5 summarizes the statistical treatment and evaluation metrics used in the study.

Treatment/Metric	**Purpose**	**Applied To**	**Decision/Interpretation Criterion**
Wilcoxon Signed-Rank Test	Determines whether a statistically significant difference exists between the two configurations	Energy consumption, Joules per token, inference speed, BERTScore F1, CPU utilization, GPU utilization, and memory usage across the 500-prompt evaluation set	Significant if $p < 0.05$
Descriptive Statistics	Summarizes the performance profile of each configuration	Mean, standard deviation, minimum, and maximum values for all dependent variables	Reported descriptively
Per-Variant Descriptive Analysis	Examines how GGUF precision/quantization variant affects	F16, Q8_0, and Q4_K_M in terms of energy consumption, inference speed,	Reported descriptively

	performance outcomes	BERTScore F1, and resource utilization	
BERTScore F1	Measures semantic similarity between generated responses and reference outputs	Output quality evaluation	Higher score indicates stronger semantic similarity
EIR – Energy Improvement Ratio	Measures energy reduction	BOPIS versus Random Search	EIR > 0 indicates energy improvement; EIR ≥ 15% is reported as a substantial improvement flag
Speed Retention Ratio (SRR)	Verifies whether inference speed is retained after optimization	BOPIS versus Random Search	SRR ≥ 95%
Quality Retention Ratio (QRR)	Verifies whether output quality is retained after optimization	BOPIS versus Random Search	QRR ≥ 98%
GP MAE	Measures the average prediction	GP-predicted energy values versus actual	Lower MAE indicates better prediction

	error of the Gaussian Process surrogate model	measured energy values across BO iterations	accuracy
GP NPE	Measures normalized GP prediction error relative to mean measured energy	Bayesian Optimization process validation	NPE < 10% is treated as a researcher-defined reliability threshold
Convergence and ΔE	Evaluates whether the optimization process improves over time	Best energy value found per BO iteration and improvement per iteration	Positive or stabilizing ΔE indicates convergence toward lower-energy configurations
Sample Efficiency Ratio (SER)	Compares how quickly BOPIS and random search identify their selected configurations	BOPIS versus random search iteration logs	SER > 1.0 indicates BOPIS found its selected configuration in fewer evaluations than random search

Table 3.5. Statistical Treatment Summary

References:

Abu-Shaira, M., & Shi, W. (2025). *Unveiling statistical significance of online regression over multiple datasets*. arXiv. https://arxiv.org/abs/2512.12787

Banner, R., Nahshan, Y., & Soundry, D. (2019). *Post training 4-bit quantization of convolutional networks for rapid-deployment. Advances in Neural Information Processing Systems, 32. https://arxiv.org/abs/1810.05723*

Bast, S., Fazlic, L. B., Naumann, S., & Dartmann, G. (2024). *LLM on the Edge: Quality, Latency, and Energy Efficiency. DI.gi.de, 1183–1192. https://doi.org/10.18420/inf2024_104*

Bergstra, J., & Bengio, Y. (2012). *Random search for hyper-parameter optimization.* Journal of Machine Learning Research, *13*, 281–305. https://www.jmlr.org/papers/volume13/bergstra12a/bergstra12a.pdf

Carrasco, J., García, S., Rueda, M. M., Das, S., & Herrera, F. (2020). *Recent trends in the use of statistical tests for comparing swarm and evolutionary computing algorithms: Practical guidelines and a critical review.* Swarm and Evolutionary Computation, 54, 100665. https://doi.org/10.1016/j.swevo.2020.100665

Chen, K., Luo, W,. Zhu, Z., Hu, Y., & Xi, Y. (2022). *BAMBO: Construct Ability and Efficiency LLM Pareto Set via Bayesian Adaptive Multi-objective Block-wise Optimization.* Arxiv.org. https://arxiv.org/html/2512.09972v2

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence

Erlbaum Associates.

Databricks. (2023). *Databricks Dolly 15k [Data Set].* Hugging Face. https://huggingface.co/datasets/databricks/databricks-dolly-15k

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research, 7*, 1–30. https://www.jmlr.org/papers/volume7/demsar06a/demsar06a.pdf

Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). *LLM.int8(): 8-bit matrix multiplication for transformers at scale*. Advances in Neural Information Processing Systems, 35.

Field, A. (2018). *Discovering statistics using IBM SPSS statistics* (5th ed.). Sage Publications.

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). *GPTQ: Accurate post-training quantization for generative pre-trained transformers.* arXiv preprint arXiv:2210.17323.

Hoxha, J., Thanasi-Boçe, M., & Khalifa, T. (2025). *A deployment-aware framework for carbon- and water-efficient LLM serving.* Sustainability*, 17*(23), 10473. https://doi.org/10.3390/su172310473

Husom, J., et al. (2024). *The price of prompting: Profiling energy use in large language model inference*.arXiv.https://arxiv.org/abs/2407.16893

Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H., & Kalenichenko, D.

(2018). Quantization and training of neural networks for efficient integer-arithmetic-only inference. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 2704–2713). https://doi.org/10.1109/CVPR.2018.00286

Jain, R. (1991). The art of computer systems performance analysis: Techniques for experimental design, measurement, simulation, and modeling. Wiley.

Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient Global Optimization of Expensive Black-Box Functions. Journal of Global Optimization, 13(4), 455-492. https://doi.org/10.1023/a:1008306431147.

Kakolyris, A. K., Masouros, D., Vavaroutsos, P., & Xydis, S. (2025). throttLL'eM: Predictive GPU throttling for energy-efficient LLM inference serving. *IEEE International Symposium on High Performance Computer Architecture (HPCA)*. https://doi.org/10.1109/HPCA61900.2025.00103

Krishnamoorthi, R. (2018). *Quantizing deep convolutional networks for efficient inference. arXiv preprint arXiv:1806.08342.* https://arxiv.org/abs/1806.08342

Lannelongue, G., Grealey, J., & Inouye, M. (2021). *Green algorithms: Quantifying the carbon footprint of computation. Advanced Science, 8(12), 2100707.* https://doi.org/10.1002/advs.202100707

Lee, J., Park, S., Kwon, J., Oh, J., & Kwon, Y. (2025). *Exploring the trade-offs: Quantization methods, task difficulty, and model size in large language models from edge to giant.*

Proceedings of the 34th International Joint Conference on Artificial Intelligence (IJCAI-25). arXiv:2409.11055.

Niu, Y., et al. (2025). *TokenPowerBench: A benchmarking framework for token-level energy measurement in large language models*.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., Avila, R., Babuschkin, I., Balaji, S., Balcom, V., Baltescu, P., Bao, H., Bavarian, M., Belgum, J., … Zoph, B. (2024, March 4). *GPT-4 technical report*. arXiv.org. https://arxiv.org/abs/2303.08774

Patterson, D., Gonzalez, J., Le, Q., Liang, C., Munguia, L., Rothchild, D., So, D., Texier, M., & Dean, J. (2021). *Carbon emissions and large neural network training.* arXiv preprint arXiv:2104.10350.

Sabbatella, A. (2025). *MALBO: Optimizing LLM-based multi-agent teams via multi-objective Bayesian optimization.* arXiv. https://arxiv.org/abs/2511.11788

Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green AI. *Communications of the ACM, 63*(12), 54–63. https://doi.org/10.1145/3381831

Shahriari, B., Swersky, K., Wang, Z., Adams, R. P. & de Freitas, N. (2016). *Taking the Human Out of the Loop: A Review of Bayesian Optimization.* Proceedings of the IEEE, 104(1), 148-175. https://doi.org/10.1109/jproc.2015.2494218

Shi, Y., & Ding, C. (2025). *Systematic Characterization of LLM Quantization: A Performance, Energy, and Quality Perspective*. arXiv preprint arXiv:2508.16712.

Snoek, J., Larochelle, H., & Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. *Advances in Neural Information Processing Systems, 25*, 2951–2959.

Tanaka, K., Ito, M., Nishimura, Y., Matsude, K., & Nakayama, A. (2026). *AE-LLM: Adaptive Efficiency Optimization For Large Language Models.* Arvix. https://arxiv.org/abs/2603.20492

Tanim, A. H., Smith-Lewis, C., Downey, A., Imran, J., & Goharian, E. (2024). Bayes_Opt-SWMM: A Gaussian process-based Bayesian optimization tool for real-time flood modeling with SWMM. *Environmental Modelling & Software, 179*, 106122. https://doi.org/10.1016/j.envsoft.2024.106122

Wan, Z., Wang, X., Liu, C., Alam, S., Zheng, Y., Liu, J., Qu, Z., Yan, S., Zhu, Y., Zhang, Q., Chowdhury, M., & Zhang, M. (2023). *Efficient large language models: A survey*. arXiv. https://doi.org/10.48550/arXiv.2312.03863

Wang, C., Liu, X., & Awadallah, A. H. (2023). *Cost-Effective Hyperparameter Optimization for Large Language Model Generation Inference.* https://doi.org/10.48550/arxiv.2303.04673

Wang, J., Du, C., Yan, F., Hua, M., Gongye, X., Quan, Y., Xu, H., & Zhou, Q. (2025). Bayesian

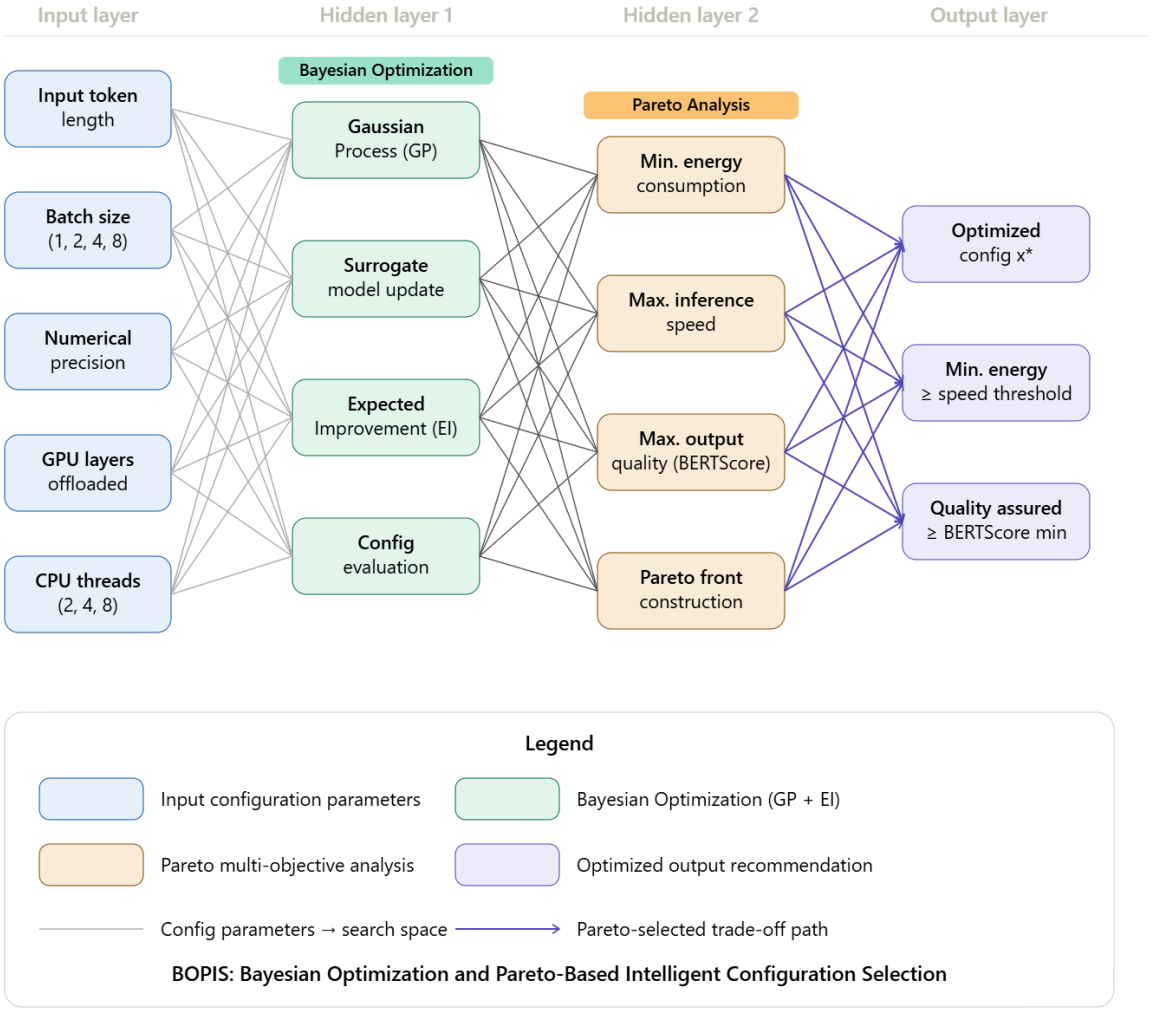
optimization for hyper-parameter tuning of an improved twin delayed deep deterministic policy gradients based energy management strategy for plug-in hybrid electric vehicles. *Applied Energy, 381*, 125171. https://doi.org/10.1016/j.apenergy.2024.125171

Wilkins, G., et al. (2024). *Offline Energy-Optimal LLM Serving: Workload-Based Energy Models for LLM Inference on Heterogeneous Systems. arXiv. https://arxiv.org/pdf/2407.04014*

Yan, B., Li, K., Xu, M., Dong, Y., Zhang, Y., Ren, Z., & Cheng, X. (2025). On protecting the data privacy of Large Language Models (LLMs) and LLM agents: A literature review. *High-Confidence Computing, 5*(2), Article 100300. https://doi.org/10.1016/j.hcc.2025.100300

Appendices:

Appendices A: BOPIS Neural Network



Appendices B: Experimentation

Trial No.	Config Type	Energy (J)	J/token	Tokens/sec	BERTScore F1	CPU Util. (%)	GPU Util. (%)	Memory (MB)
1	Random Search							

2	Random Search							
3	BOPIS							
…	…							
30	BOPIS							

Table B.1. Pre-Experiment Instrument Template for Performance Comparison

Config ID	Token Length	Batch Size	Precision	GPU Layers Offloaded	CPU Threads	Energy (J)	Tokens/sec	BERTScore F1
C1	128	1	F16	0	2			
C2	256	2	F16	14	4			
C3	512	4	INT8	28	8			
…	…	…	…	…	…			
C(n)	1024	8	F16	All	All			

Table B.2. Pre-Experiment Instrument Template for Configuration Evaluation

	Random Search		BOPIS		Flag
Prompt #	Total J	J/Token	Total J	J/Token	> 15%?
1					
2					
3					
…					

498					
499					
500					
Mean					
SD					

Table B.3. Energy Consumption Per Prompt

Prompt #	Task Type	Random Search (token/s)	BOPIS (token/s)	BOPIS Config Selected	$S \geq S_{min}$?
1					
2					
3					
…					
498					
499					
500					
Mean					
SD					

Table B.4. Inference Speed per Prompt

Prompt #	Task Type	Random F1	BOPIS F1	$Q_{min}(task)$	$Q \geq Q_{min}$?
1					
2					
3					
…					
498					

499					
500					
Mean					
SD					

Table B.5. Output Quality per Prompt (BERTScore F1)

	Random Search			BOPIS		
Prompt #	**CPU%**	**GPU%**	**RAM MB**	**CPU%**	**GPU%**	**RAM MB**
1						
2						
3						
…						
498						
499						
500						
Mean						
SD						

Table B.6. Resource Utilization Per Prompt

Iter.	**Config x** (t, b, p, g, c)	**E measured (J)**	**S** (tok/s)	**Q** $(F1)$	$\mu(x)$ **GP pred.**	$\sigma(x)$ **Uncertainty**	$EI(x)$ **Acquisition**	**Pareto Front?**
1 (random)								
2 (random)								
3 (BO)								
4 (BO)								

…								
N (BO)								
$C_{boTotal}$		**Sum J**						**Front Size**

Table B.7. BOPIS Calibration Log

DependentVariable	Statistic	Random Search	BOPIS
Energy -- Total (J)	*Mean*		
	SD		
	Min		
	Max		
Energy -- J/token	*Mean*		
	SD		
	Min		
	Max		
Inference Speed (tok/s)	*Mean*		
	SD		
	Min		
	Max		
Output Quality (BERTScore F1)	*Mean*		
	SD		
	Min		
	Max		
CPU Utilization (%)	*Mean*		
	SD		
	Min		

	Max		
GPU Utilization (%)	*Mean*		
	SD		
	Min		
	Max		
Memory Usage (MB)	*Mean*		
	SD		
	Min		
	Max		
Wilcoxon Signed-Rank	*p-value*	RS vs BOPIS	BOPIS vs RS

Table B.8. Summary Comparison

Appendix C: First Iteration of System Architecture

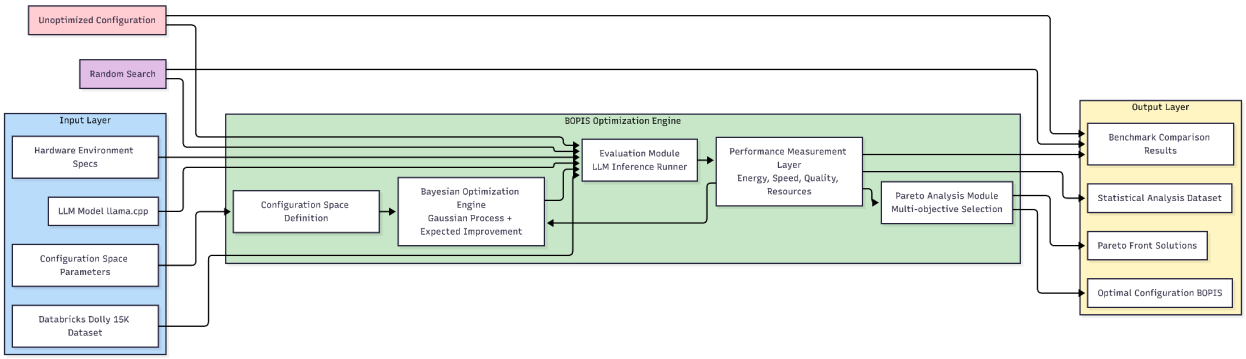


Figure A.6. First Iteration of System Architecture

Appendix D: Second Iteration of System Architecture

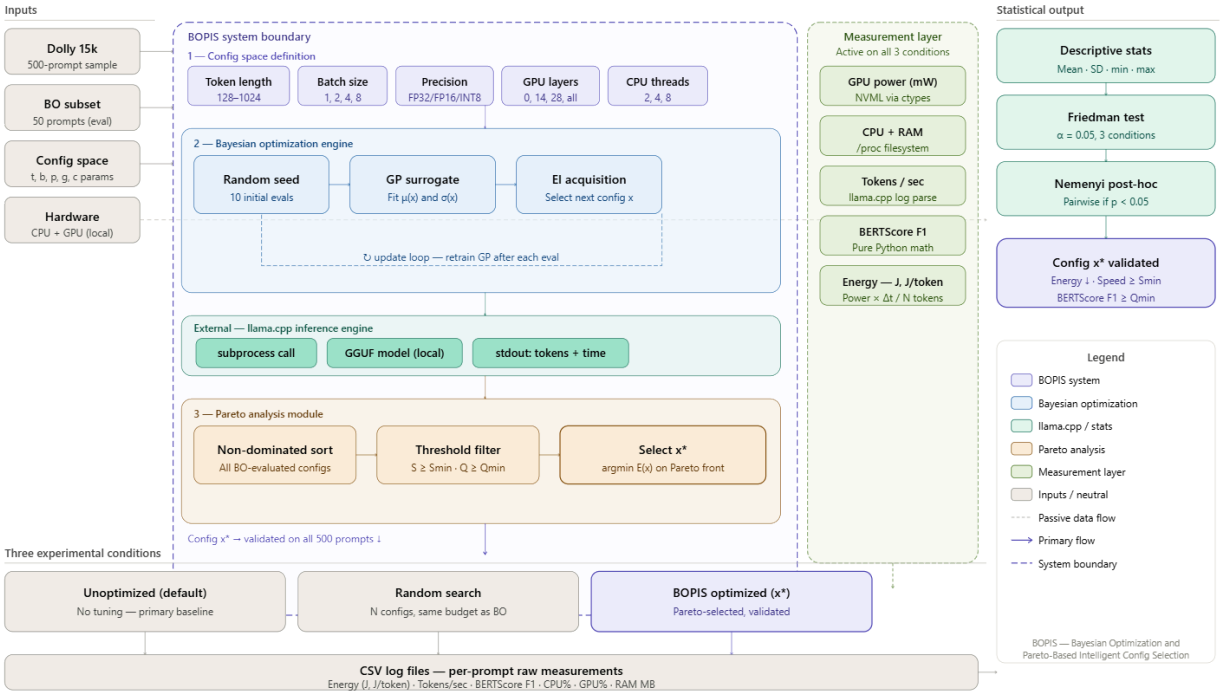


Figure A.7. Second Iteration of System Architecture